เตรียม ตอบ คำถาม Colorado Back-Office System

01418321 System Analysis and Design หมู่ 200
ระบบจัดการหลังบ้านเฟส 1 Rakuten Ichiba, Rakuten RMS และ RSL

หมวดคำถาม

หมวด	สไลด์ / หัวข้อในเล่ม	ผู้ตอบหลัก	จำนวน
1. ธุรกิจ ปัญหา ขอบเขต	สไลด์ 3–6 / 1.1–1.5	ธนภัทร	8
2. Business Process	สไลด์ 7–11 / 2.1–2.7	ธนภัทร	6
3. Use Case, Actor, CRUD	สไลด์ 12–14 / 2.8–2.12	ธนภัทร	8
4. Use Case Description	สไลด์ 15–33 / 2.13	อชิระ	7
5. Sequence, Collaboration	สไลด์ 17–33 / 2.13	อชิระ	6
6. ER และโครงสร้างตาราง	สไลด์ 34–36 / 2.14–2.17	ปิยากร	6
7. Class, State Diagram	สไลด์ 37–43 / 2.18–2.19	ปิยากร	4
8. สถาปัตยกรรมเว็บจริง	สไลด์ 44–48 / 2.18, 2.20	ปิยากร	7
9. Platform, Site map, รายงาน	สไลด์ 49–51 / 2.20–2.22	ปิยากร	5
10. SQL	สไลด์ 36, 52 / 2.16, 2.23	ปิยากร	3
11. การทดสอบและเดโม	สไลด์ 53–54 / 2.24	ปิยากร	5
12. ทำไมไม่ทำแบบอื่น ข้อจำกัด	สไลด์ 56	ทุกคน	8
13. คำถามถึงตัวบุคคล	สไลด์ 55	ทุกคน	4
		รวม	84

ผู้ตอบหลักตามช่วงสไลด์ที่แต่ละคนพูด ธนภัทร 1–14, อชิระ 15–33, ปิยากร 34–57

วิธีใช้เล่มนี้

- หัวข้อ **ตอบ** เขียนเป็นประโยคที่พูดต่อกันได้ ส่วน**วิธีตอบ**เป็นคำแนะนำ ไม่ต้องพูดออกมา
- ข้อที่มีดาว ★ มีโอกาสโดนถามสูง ทุกคนต้องตอบได้ ไม่ใช่แค่ผู้ตอบหลัก
- ทุกคำตอบอิงโค้ดจริงที่ branch `dev` (commit `8c4c7ef` รวม PR #28 และ #29) และรายงานฉบับล่าสุด 297 หน้า ถ้าเล่มกับโค้ดขัดกัน ให้ยึดโค้ด
- เลข Q เรียงต่อกันทั้งเล่ม ใช้สุ่มเลขตอนซ้อมได้ ท้ายเล่มมีตารางว่าเกณฑ์ 36 ข้อแต่ละข้อน่าจะโดนถามที่ Q ไหน

ก่อนเริ่ม

อ่านทุกคน

เรื่องที่ต้องตกลงกันก่อน และวิธีตอบ

ต้องตกลงกันก่อนวันนำเสนอ

1. **หัวหน้าทีมและการแบ่งงาน** ตาราง 2.25 ต้องมีชื่อหัวหน้าและคะแนนการทำงาน (Q74)
2. **รหัสผ่านบัญชีเดโม** มาจาก `SEED_ADMIN_PASSWORD` ใน `.env.local` คนเดโมต้องรู้ แต่ห้ามขึ้นสไลด์ (Q64)

ตอบ 4 จังหวะ จบใน 30 ถึง 45 วินาที

1. **ฟังให้จบ** แล้วทวนคำถามสั้น ๆ
2. **ตอบตรง ๆ** ประโยคแรกคือคำตอบ
3. **ให้หลักฐาน** เปิดหน้าในเล่ม เปิดสไลด์ หรือเปิดเว็บ 1 อย่าง
4. **โยงกลับ** เชื่อมกับ human-in-the-loop หรือปัญหาของเจ้าของ

ประโยคที่ใช้ได้เลย

- **ไม่รู้คำตอบ** “เรื่องนี้เรายังไม่ได้ออกแบบไว้ครับ/ค่ะ แต่ถ้าจะทำ จะเพิ่มที่...” ห้ามเดาตัวเลขหรือแต่งว่าทำไปแล้ว
- **ถูกชี้จุดผิด** “ขอบคุณครับ/ค่ะ จุดนี้เราพลาด จะแก้ให้ตรงกันในเล่มฉบับส่ง” ยอมรับแล้วบอกวิธีแก้ ดีกว่าเถียง
- **ถามหลายข้อพร้อมกัน** “ขอตอบทีละข้อนะครับ/คะ” แล้วจดไว้
- **นอกขอบเขต** “เรื่องนี้อยู่นอกเฟส 1 ที่ตกลงกับเจ้าของไว้ แต่ออกแบบให้ต่อได้ที่...”
- **เพื่อนตอบไม่ครบ** “ขอเสริมนิดหนึ่งครับ/ค่ะ...” แล้วพูดแค่ส่วนที่ขาด

ห้ามพูด

- “เจ้าของใช้เวลา 2 ชั่วโมงต่อวัน” แบบไม่มีคำว่าประมาณ ตัวเลขเวลาทั้งหมดเป็น**ค่าประมาณ**จากขั้นตอนที่เจ้าของเล่า
- “ระบบสั่งซื้อให้อัตโนมัติ” ที่ถูกคือระบบ**เตรียม**ข้อมูลให้ แต่เจ้าของต้องกดตัดสินใจใน 5A และกดสั่งซื้อใน 6A เสมอ
- “เว็บเก็บข้อมูลใน localStorage” หรือ “มี 15 ตาราง 63 SQL” เป็นข้อมูลรุ่นเก่า ตอนนี้เว็บอ่านเขียน **PostgreSQL จริง 17 ตาราง** และ SQL ในเล่ม **113 ข้อ**
- “มี Controller, Repository, Scheduler” โค้ดจริงไม่มีคลาสเหล่านี้ ชั้นจริงคือ หน้าจอ → Server Action → use-case → queries → db.ts
- “เชื่อม Rakuten RMS และ RSL แล้ว” ที่ถูกคือเรียกผ่าน adapter ที่ตอนนี้เป็น**ตัวจำลอง (mock)** ทั้ง 6 ตัว

1. ธุรกิจ ปัญหา ขอบเขต

8 ข้อ

สไลด์ 3–6 ในเล่ม บทคัดย่อ และ 1.1–1.5

ผู้ตอบหลัก นายธนภัทร พูลโภคา

Q1 Please introduce the business in English ★ ทุกคนต้องตอบได้

วิธีตอบ เกณฑ์การนำเสนอกำหนดให้แนะนำธุรกิจเป็นภาษาอังกฤษ ถ้าอาจารย์ขอให้พูดซ้ำหรือถามต่อเป็นภาษาอังกฤษ ใช้ 4 ประโยคนี้ พูดช้า ๆ

ตอบ

- “Colorado Company Limited sells imported and OEM products on Rakuten Ichiba in Japan, such as fishing lures, outdoor goods and Thai herbal inhalers.”
- “It started in 2009 as a side business and is run by one owner. Rakuten Super Logistics, or RSL, stores and ships the parcels.”
- “Every order needs the same manual steps: check stock in two warehouses, match the order with RSL, print the label and send the tracking number. We estimate one to two hours a day.”
- “Our system automates these repeated steps, but the owner still approves every purchase that spends money.”

เปิดหลักฐาน สไลด์ 3 (บทพูดอยู่ใน notes ของสไลด์), บทคัดย่อภาษาอังกฤษ

Q2 ทำไมเฟส 1 ทำแค่ Rakuten ★ ทุกคนต้องตอบได้

ตอบ

- Rakuten Ichiba เป็นช่องทางขายหลัก มีคำสั่งซื้อจริงให้ทดสอบได้ทันที
- ร้านใช้ RSL เป็นคลังหลักอยู่แล้ว ซึ่งอยู่ในระบบเดียวกับ Rakuten จึงเชื่อมคำสั่งซื้อ สต๊อก และใบปะได้ต่อเนื่อง
- Rakuten RMS มี API ครอบคลุมทั้งดึงคำสั่งซื้อ รูปแบบใบปะ และสต๊อก ตรงกับ 3 งานหลัก
- ทำทีละแพลตฟอร์มลดความเสี่ยงจากการเชื่อมหลายระบบพร้อมกัน Yahoo! Auctions กับ Amazon อยู่เฟสถัดไป

เปิดหลักฐาน รายงาน 1.1 และ 1.5

Q3 ตัวเลข 1 ถึง 2 ชั่วโมงต่อวันมาจากไหน ★ ทุกคนต้องตอบได้

ตอบ

- มาจากจำนวนขายประมาณ **5 ถึง 10 ชิ้นต่อวัน** ตาม requirement คูณเวลาต่อคำสั่งซื้อประมาณ **12 นาที**
- 12 นาทีแยกเป็น 5 งาน ตรวจคำสั่งซื้อ 2 เช็กสต๊อก 3 จับคู่ RSL 2 จัดรูปแบบและพิมพ์ใบปะ 3 แจ้งเลข

ติดตาม 2 นาที

- ได้ **60 ถึง 120 นาทีต่อวัน** หรือ 30 ถึง 60 ชั่วโมงต่อเดือน ยังไม่รวมการคำนวณต้นทุน 20 ถึง 30 นาทีต่อครั้ง
- ตัวเลขเวลาเป็นค่าประมาณจากขั้นตอนที่เจ้าของเล่าตอนสัมภาษณ์ ยังไม่ได้จับเวลาจริง

เปิดหลักฐาน ตารางที่ 1-1 ในหัวข้อ 1.2, สไลด์ 4

ระวัง ตัวเลขที่ได้มาจากเจ้าของโดยตรงคือ 5–10 ชิ้นต่อวัน ค่าส่ง 600 กับ 200 เยน และค่า RSL 60 เยน ส่วนนาทีต่องานเป็นของเราประมาณเอง

Q4 ปัญหาไม่ใช่แค่ ``ไม่มีคอมพิวเตอร์'' หรือ

ตอบ

- ไม่ใช่ครับ/ค่ะ เจ้าของใช้คอมพิวเตอร์และ Rakuten RMS อยู่แล้ว
- ปัญหาคือข้อมูลอยู่คนละที่ ต้องคัดลอกและเทียบเองทุกคำสั่งซื้อ เช่น สต๊อกอยู่ทั้งคลังบริษัทและ RSL ต้นทุนต้องรวม 8 ค่าเอง
- และตัดสินใจสั่งซื้อด้วยความรู้สึก ไม่มีจุดสั่งซื้อที่ชัดเจน
- ทุกปัญหาวัดได้ว่าลดลงหรือไม่ เช่น เวลาต่อวัน และเวลาคำนวณต้นทุน

เปิดหลักฐาน รายงาน 1.2 และ 2.3

Q5 วัดอย่างไรว่าระบบแก้ปัญหาได้จริง

ตอบ

ตั้งตัวชี้วัดไว้ภายใน 3 เดือนหลังเปิดใช้งาน 4 ข้อ

- คำสั่งซื้อ Rakuten ใหม่**ทุกรายการ**ได้ใบปะที่ถูกต้องโดยไม่ต้องจัดรูปแบบเอง
- สต๊อกที่ต่ำกว่าเกณฑ์ถูกแจ้งเตือนครบทุก SKU
- คำนวณต้นทุนต่อหน่วยได้ภายใน **1 นาที** แทน 20 ถึง 30 นาที
- เวลางานหลังบ้านต่อวันลดลง**อย่างน้อยครึ่งหนึ่ง**จากตารางที่ 1-1

เปิดหลักฐาน รายงาน 1.3, สไลด์ 5

Q6 ขายแค่ 5 ถึง 10 ชิ้นต่อวัน คุ้มไหมที่จะทำระบบ

ตอบ

- งานมือ 1 ถึง 2 ชั่วโมงต่อวัน คือ 30 ถึง 60 ชั่วโมงต่อเดือน ของเจ้าของที่ทำทุกอย่างคนเดียว เวลานี้ควรไปพัฒนาสินค้าใหม่และเจรจาโรงงาน
- เรื่องเงินก็มีผล ส่งเองประมาณ **600 เยน** ต่อชิ้น คู่แข่งประมาณ 200 เยน ต่างกัน 400 เยน หรือ 2,000 ถึง 4,000 เยนต่อวัน ขณะที่ RSL คิดค่าหยิบประมาณ 60 เยน ต้องมีต้นทุนต่อหน่วยที่ถูกต้องจึงเลือกวิธีส่งและตัดสินใจสั่งซื้อได้

- ร้านจะขยายไป Yahoo! Auctions และ Amazon งานจะเพิ่มขึ้นตามช่องทาง ระบบจึงวางโครงไว้ตั้งแต่ตอนนี้

เปิดหลักฐาน รายงาน 1.2 และ 1.7, สไลด์ 4

Q7 ทำไมไม่ให้ระบบสั่งซื้อจาก Supplier เองเลย ★ ทุกคนต้องตอบได้

ตอบ

- เพราะเป็นขั้นที่ใช้เงินจริง ถ้าข้อมูลสต๊อกหรือต้นทุนผิด ระบบอาจสั่งของผิดจำนวนโดยไม่มีใครเห็น
- อาจารย์แนะนำให้ใช้ human-in-the-loop ระบบตรวจสต๊อก (3S) คำนวณต้นทุน (4S) และแสดงกำไรต่อหน่วยที่คาด แต่ 5A ให้เจ้าของกด “คุ้มค่า” หรือ “ไม่คุ้มค่า” เอง และใน 6A เจ้าของต้องกดปุ่มสั่งซื้อ
- ใน Use Case Diagram มีเส้นประสีแดง “ต้องอนุมัติ (5A) ก่อน” ระหว่าง 5A กับ 6A

เปิดหลักฐาน สไลด์ 6 และ 10, รายงาน 2.5 ขั้นตอนที่ 13, Use Case Description 5A

ระวัง requirement ข้อ 5 เขียนว่า Auto Reorder ส่งคำสั่งซื้อ**โดยอัตโนมัติ** ถ้าถูกถามว่าขัดกับ requirement ไหม ให้ตอบว่า requirement ข้อเดียวกันเขียนว่า “การอนุมัติขั้นสุดท้าย” ยังทำด้วยมือ ส่วนที่อัตโนมัติคือการเช็กสต๊อกและเตรียมข้อมูลการสั่งซื้อ

Q8 อะไรที่ยังทำด้วยมือ และอะไรอยู่นอกขอบเขต

ตอบ

- ยังทำด้วยมือ วางแผนสินค้า จัดหา เจรจาโรงงาน ตัดสินใจเรื่องคุณภาพ อนุมัติการสั่งซื้อ กดรับสินค้าเมื่อของมาถึง กดพิมพ์ใบปะ และส่งมอบพัสดุให้ขนส่ง
- นอกขอบเขตเฟส 1 Yahoo! Auctions, Amazon SP-API, การสื่อสารกับลูกค้าโดยตรง การรับชำระเงิน และ 2FA ส่วน Mercari Shops เลิกใช้แล้ว
- ผู้ใช้หลักคือเจ้าของในฐานะ Super Admin ลูกค้ายังสั่งและจ่ายเงินผ่าน Rakuten ตามเดิม ไม่มีหน้าร้าน

เปิดหลักฐาน รายงาน 1.5

2. Business Process

6 ข้อ

สไลด์ 7–11 ในเล่ม 2.1–2.7

ผู้ตอบหลัก นายธนภัทร พูลโภคา

Q9 อธิบาย Business Process เดิมตั้งแต่ต้นจนจบ

ตอบ

- มี 4 lane คือ Marketplace, Super Admin, Delivery และ Supplier โดย Super Admin กับเจ้าของคือคนเดียวกัน
- ลูกค้าสั่งและจ่ายเงินที่ Marketplace แล้ว Super Admin ตรวจคำสั่งซื้อเอง เช็กว่าสต๊อกพอไหม
- ถ้าพอ จับคู่เลข RSL กับ order_id เอง พิมพ์ใบปะเอง ส่งให้ Delivery แล้วส่งเลขติดตามกลับไปที่ Marketplace เอง
- ถ้าไม่พอ คำนวณต้นทุนเองว่าคุ้มไหม ถ้าคุ้มสั่ง Supplier ถ้า Supplier มีของ ส่งของเข้าร้านแล้ววนกลับไปจับคู่ RSL ถ้าไม่คุ้มหรือ Supplier ไม่มีของ ยกเลิกคำสั่งซื้อ

เปิดหลักฐาน รูป 2.1 และคำอธิบาย 2.2.1–2.2.15, สไลด์ 7

Q10 จุดที่เป็นปัญหาในรูปมีกี่จุด แต่ละจุดเสียเวลาเท่าไร

ตอบ

มี 4 จุดสีแดง แต่ละจุดผูกกับเวลาประมาณในตารางที่ 1-1

- 2.3.1 จับคู่ Order กับสต๊อกเอง ประมาณ **5 นาที** ต่อ Order (เช็กสต๊อก 3 จับคู่ RSL 2)
- 2.3.2 คำนวณความคุ้มค่าเอง รวม 8 ค่า ประมาณ **20 ถึง 30 นาที** ต่อครั้ง
- 2.3.3 จัดรูปแบบใบปะเอง ประมาณ **3 นาที** ต่อ Order
- 2.3.4 ติดตามพัสดุและแจ้งลูกค้าเอง ประมาณ **2 นาที** ต่อ Order

เปิดหลักฐาน รูป 2.3 และหัวข้อ 2.3, สไลด์ 8

Q11 Business Process ใหม่ต่างจากเดิมตรงไหน ★ ทุกคนต้องตอบได้

ตอบ

- เพิ่ม lane “ระบบ (System)” รับขั้นที่ทำซ้ำได้ 7 Use Case คือ 1S ถึง 7S เช่น เช็กสต๊อก จับคู่ RSL จัดรูปแบบใบปะ แจ้งเลขติดตาม ตัดสต๊อก
- หลังเช็กสต๊อก (3S) มี 3 ทาง RSL พอ → จับคู่ RSL (1S), RSL ไม่พอแต่คลังบริษัทพอ → ข้ามไปจัดรูปแบบใบปะ (6S) เลย, ไม่มีแหล่งใดพอ → คำนวณต้นทุน (4S) และตัดสินใจ (5A)
- คนยังทำขั้นที่ต้องตัดสินใจหรือใช้มือ เช่น ตรวจคำสั่งซื้อ (2A) ตั้งกฎ SKU (3A) ตัดสินใจ (5A) สั่งซื้อและรับสินค้า (6A) ยกเลิก (7A) ส่งมอบพัสดุ (8A) พิมพ์ใบปะ (9A)
- ตาราง 2.5 มีคอลัมน์ “แก้ปัญหาข้อ” บอกว่าขั้นไหนแก้ปัญหาข้อไหนใน 2.3 และเพิ่ม 10A ลบข้อมูลเก่าตามคำแนะนำของอาจารย์

เปิดหลักฐาน รูป 2.4 และตาราง 2.5, สไลด์ 9–10

Q12 requirement บอกว่าพิมพ์ใบปะอัตโนมัติ ทำไม 9A ยังให้คนกด

ตอบ

- ส่วนที่เสียเวลาคือการจัดรูปแบบ ซึ่งระบบทำให้แล้วใน 6S ทันทีหลัง 1S จับคู่ RSL หรือหลัง 3S เลือกคลังบริษัท เลือก template ตามแหล่งจัดส่งและวิธีส่งเอง (RSL-STD-A6, RSL-LARGE-A5, INHOUSE-YAMATO-A6)
- การพิมพ์เป็นขั้นทางกายภาพ ต้องมีคนใส่กระดาษ ติดใบปะ และเตรียมพัสดุอยู่แล้ว จึงเหลือแค่กดพิมพ์
- ถ้าพิมพ์เองโดยไม่มีคนอยู่ เครื่องพิมพ์ไม่พร้อมจะไม่มีใครเห็น ใน 9A จึงมีทางเลือกกรณีเครื่องพิมพ์ไม่พร้อมและพิมพ์ซ้ำ ทุกครั้งบันทึกลง label_print_history

เปิดหลักฐาน ตาราง 2.5 ขั้นตอนที่ 7–8, Use Case Description 6S และ 9A, สไลด์ 23

Q13 แปลง Business Process เป็น Use Case อย่างไร

ตอบ

- นำรูป BP ใหม่มาเขียนเลข Use Case กำกับทุกกล่อง ทั้งกล่องที่ระบบทำ (S) และกล่องที่คนทำ (A) ได้รูป 2.6
- ตาราง 2.7 จับคู่ทีละขั้น 16 แถว ทุก Use Case มาจากกล่องใน BP
- กล่องที่เป็นจุดตัดสินใจ เช่น “มีกฎ SKU ที่ตรงกันในระบบไหม” กลายเป็น 2S

เปิดหลักฐาน รูป 2.6 และตาราง 2.7, สไลด์ 11

Q14 ทำไม 1A และ 10A ไม่อยู่ใน Business Process

ตอบ

- 1A เข้าสู่ระบบเป็นเงื่อนไขก่อนเริ่มทุกขั้น ไม่ใช่ขั้นหนึ่งในการรับส่งคำสั่งซื้อ
- 10A เป็นงานดูแลระบบที่ทำเป็นระยะ ไม่ได้ผูกกับคำสั่งซื้อใดคำสั่งซื้อหนึ่ง
- เขียนเหตุผลไว้ใต้ตารางจับคู่ BP กับ UC ส่วน 10A ยังมีในตาราง BP ใหม่ขั้นตอนที่ 16

เปิดหลักฐาน หัวข้อ 2.7 (ใต้ตาราง), สไลด์ 11

3. Use Case, Actor, CRUD

8 ข้อ

สไลด์ 12–14 ในเล่ม 2.8–2.12

ผู้ตอบหลัก นายธนภัทร พูลโภคา

Q15 เลข Use Case และเลข query อ่านอย่างไร ★ ทุกคนต้องตอบได้

ตอบ

- เลขตามด้วย A คือ Admin เป็นผู้เริ่ม มี 10 ตัว 1A–10A ส่วน S คือระบบทำเอง มี 7 ตัว 1S–7S รวม **17 Use Case**
- query ใช้รูปแบบ Q ตามด้วยเลข Use Case และลำดับในขั้นตอน เช่น Q6A.11 คือ query ที่ 11 ของ 6A (บันทึกรับสินค้า) รวมทั้งเล่ม **113 query**
- เลขเดียวกันใช้ใน description, Sequence, Collaboration, ER พร้อม Query, Query Table และผล SQL
- ในโค้ด SQL เก็บเป็นค่าคงที่ชื่อ `Q<UC>_<n>_...` ใน `src/lib/queries` ชื่อในโค้ดไม่จำเป็นต้องตรงเลขในเล่ม เพราะ query เดียวใช้ซ้ำหลาย Use Case Query Table 2.16 มีคอลัมน์บอกชื่อค่าคงที่ในโค้ดของทุกข้อ

ระวัง Use Case Diagram ก่อนปรับปรุงมีเลขชุดของตัวเอง เช่น 1A เดิมคือ “ตรวจสอบ Order แบบ manual” แต่ 1A ใหม่คือเข้าสู่ระบบ ถ้าถูกถามให้บอกว่าเรียงเลขใหม่หลังทบทวน

เปิดหลักฐาน Query Table 2.16, สไลด์ 15 และ 36

Q16 Actor มีใครบ้าง RSL เป็น Actor ไหม

ตอบ

- Actor ที่ใช้ระบบคือเจ้าของในฐานะ **Super Admin** คนเดียว
- Actor ภายนอกมี 4 ตัว คือ Marketplace (Rakuten Ichiba ผ่าน RMS), Supplier, Delivery และ **RSL** เพราะระบบส่งข้อมูลไปจับคู่ (1S) และอ่านยอดสต๊อกจาก RSL (3S) โดยตรง
- “ระบบ” ไม่ใช่ Actor ใน Use Case Diagram เพราะกรอบของ diagram คือระบบเองแล้ว Use Case แบบ S จึงไม่มีเส้นจาก Super Admin

เปิดหลักฐาน รายงาน 2.11 (รูป 2-6 และรายการ Actor), สไลด์ 13

Q17 Use Case ที่เพิ่มจากการวิเคราะห์ CRUD มีอะไรบ้าง ★ ทุกคนต้องตอบได้

ตอบ

- ก่อนปรับปรุงมี **9 Use Case** หลังทบทวนมี **17** โดยเพิ่ม **7 ตัว** คือ 2A ตรวจสอบคำสั่งซื้อ, 3A ตั้งกฎ SKU, 9A พิมพ์ใบปะ, 10A ลบข้อมูลเก่า, 2S จับคู่กฎ SKU, 5S ส่งเลขติดตาม, 7S ตัดสต๊อก
- เหตุผลมาจาก 2.10 ไม่มีกฎ SKU (2.10.1) งานอัตโนมัติยังผูกกับคน (2.10.2) ไม่มีขั้นตัดสต๊อก

(2.10.3) และไม่มีการลบข้อมูลเก่า (2.10.4)

- ชุดนี้ตรงกันทั้ง usecases.json (ช่อง origin = เพิ่มใหม่), รายงาน 2.12 และแถวสีแดงในสไลด์ 14

เปิดหลักฐาน รายงาน 2.10 และ 2.12, สไลด์ 12–14

Q18 Actor ของ 3S และ 4S เป็น Admin หรือระบบ

ตอบ

- ตามรหัส S ระบบเป็นผู้ทำ 3S รันเองใน `executeWorkflow` ต่อจาก 2S ทันที (และหลังรับสินค้าใน 6A) แล้วแยกเส้นทางตามสต๊อก Admin มีหน้า StockCheckScreen ไว้ดูและปรับยอดสต๊อก
- 4S คำนวณเมื่อ Super Admin กด “คำนวณ” ที่หน้า `/products/cost` หรือเมื่อบันทึกสินค้าพร้อมองค์ประกอบต้นทุนใน 3A ขั้นการคำนวณทั้งหมดระบบทำ
- 1S, 4S และ 5S เริ่มจากการกดปุ่ม แต่ทุกขั้นหลังจากนั้นระบบทำเอง จึงไม่วาดเส้นจาก Super Admin (เขียนไว้ใน 2.11)

เปิดหลักฐาน usecases.json ช่อง actor, รายงาน 2.11, สไลด์ 15

ระวัง ถ้าถูกถามว่า “4A ทำไมเป็น A” ตอบว่าในเว็บ Admin กดนำเข้า Order ที่ Dashboard เอง ไม่มีการดึงตามรอบเวลา จึงเป็น A

Q19 Use Case Diagram ก่อนปรับปรุงได้มาอย่างไร

ตอบ

- แปลงจากกล่องใน BP เดิม ชื่อ Use Case ตรงกับชื่อกล่อง ได้ 9 Use Case ที่เจ้าของทำเองเกือบทั้งหมด
- Actor วางตาม swim lane ของ BP คือ Super Admin, Marketplace, Delivery และ Supplier

เปิดหลักฐาน รูป 2.8, สไลด์ 12

Q20 CRUD Table สร้างอย่างไร entity มาจากไหน

ตอบ

- ตารางก่อนปรับปรุง (2.9) ได้จากการหาคำนามในขั้นตอนทำงาน 9 Use Case
- ตารางที่ทบทวนแล้ว (2.12) สร้างจาก SQL ทุกข้อใน description แปลง INSERT, SELECT, UPDATE, DELETE เป็น C, R, U, D ได้ 17 Use Case คูณ **17 entity** ชื่อตรงกับตารางใน ER และฐานข้อมูลจริงทุกตัว
- ข้อดีคือ CRUD ไม่มีทางหลุดจาก SQL เพราะสร้างจากที่เดียวกัน (`npm run dict` สร้าง crud.json)

เปิดหลักฐาน ตาราง 2.9 และ 2.12, ไฟล์ sql/crud.json, สไลด์ 14

Q21 บางตารางไม่มี Use Case ไหน Create เลย ใครเพิ่มข้อมูล

วิธีตอบ เป็นคำถามที่อาจารย์ใช้ CRUD Table จับจุดขาดได้ง่าย ตอบตรง ๆ ว่าเห็นแล้ว และบอกว่าจะเพิ่มอะไร

ตอบ

- suppliers มีแค่ R และ admins มีแค่ R กับ U (นับครั้งผิด ล็อกบัญชี เวลาเข้าสู่ระบบล่าสุด) เพราะเฟส 1 ตั้งค่าไว้ครั้งเดียวผ่านข้อมูลตั้งต้น (`npm run db:reset`)
- ตารางประวัติ เช่น auto_error_logs, stock_adjustment_history, label_print_history มีแค่ C เพราะเป็น log ที่เขียนอย่างเดียว
- เรายอมรับว่าเป็นช่องว่าง ถ้าทำต่อจะเพิ่ม Use Case “จัดการ Supplier” และ “จัดการบัญชีผู้ใช้” ตาม requirement ที่บอกว่าเจ้าของเป็นผู้กำหนดกฎ supplier

เปิดหลักฐาน ตาราง 2.12 (คอลัมน์ suppliers, admins)

Q22 ทำไมต้องมี 10A ลบข้อมูล ลบแล้วข้อมูลหายไหม

ตอบ

- อาจารย์แนะนำเรื่อง data retention ไม่ให้ฐานข้อมูลโตไปเรื่อย ๆ โดยไม่มีประโยชน์
- ลบได้เฉพาะข้อมูลที่ปิดแล้ว คือ Order “จัดส่งสำเร็จ” หรือ “ยกเลิกแล้ว” ประวัติต้นทุน ใบปะ และคำสั่งซื้อเติมสต๊อกที่ยกเลิก ได้รับของแล้ว หรือเลยวันรับและไม่มี Order ที่ยังเปิดอ้างถึง แบบตามช่วงวันที่ต้องเก่ากว่า **12 เดือน**
- Order ที่ลบเก็บ snapshot (Order + รายการสินค้า + ใบปะ) ลง `deletion_history` ดาวน์โหลด CSV สำรองก่อนลบได้ ทั้งชุดทำใน transaction เดียว ถ้าพลาด rollback ทั้งชุด

เปิดหลักฐาน Use Case Description 10A, สไลด์ 14 (แถว 10A มีตัว D) และ 31

4. Use Case Description

7 ข้อ

สไลด์ 15–33 ในเล่ม 2.13

ผู้ตอบหลัก นายอชิระ ภู่รุ่งเรือง

Q23 อธิบาย Use Case Description ของ 5A ★ ทุกคนต้องตอบได้

ตอบ

- 5A เป็นจุด human-in-the-loop ของระบบ Pre-condition คือ Order อยู่ที่ “รอ Admin ตัดสินใจสั่งซื้อ” (3S หาแหล่งที่พอไม่ได้) และ 4S คำนวณ unit_cost แล้ว
- หน้าจอแสดงต้นทุนต่อหน่วย ราคาขาย กำไรต่อหน่วยที่คาด (selling_price − unit_cost) และ lead time ของ Supplier
- กดยืนยันแล้ว ทำใน transaction เดียว ตรวจจำนวนเป็นจำนวนเต็มบวก แล้ว **Q5A.1** ล็อก Order (FOR UPDATE) และอ่านราคาขายกับต้นทุนล่าสุด **Q5A.2** บันทึก reorder_decision, decided_qty และสถานะใหม่ คุ้มค่า → “รอสั่งซื้อจาก Supplier” (6A) ไม่คุ้มค่า → “รอยกเลิก Order” (7A)
- ถ้าไม่มีราคาขาย **Q5A.3** พัก Order ไว้ที่ “รอดำเนินการด้วยตนเอง” (manual_reason = price) พร้อมลิงก์ไปตั้งราคาใน 3A
- ตัวอย่าง FSH-RDS-180 ราคาขาย 2,690 ต้นทุน 2,462.20 กำไรที่คาด **227.80 บาท** ต่อชิ้น

เปิดหลักฐาน สไลด์ 25–27, รายงาน 2.13 (5A)

> **ระวัง** 5A มีแค่ 3 query คือ Q5A.1–Q5A.3 เลขอย่าง Q5A.4, Q5A.7, Q5A.9 เป็นของรุ่นเก่า ไม่มีแล้ว

Q24 ทางเลือก (alternative) ใน description เขียนอย่างไร

ตอบ

แต่ละ Use Case มีทางเลือก 3 กลุ่ม

- ข้อมูลไม่ครบหรือไม่ถูกต้อง เช่น 7A ไม่ใส่เหตุผล ระบบแจ้ง “กรุณาระบุเหตุผลการยกเลิก” หรือ 5A จำนวนไม่ใช่จำนวนเต็มบวก
- ทำซ้ำหรือสถานะไม่ตรง เช่น 5A Order ถูกตัดสินใจไปแล้ว, 6A รับสินค้าซ้ำได้ “already” และ 7S ตัดเฉพาะรายการที่ stock_deducted = FALSE
- ระบบภายนอกล่ม เช่น 1S เชื่อม RSL ไม่ได้ คงสถานะเดิม, 3S แสดงเฉพาะสต๊อกคลังบริษัท, 7S อัปเดต Rakuten ไม่ได้ ตัดสต๊อกในระบบไว้ก่อนแล้วตั้ง sync เป็น pending ให้ลองใหม่

กรณีที่ระบบตัดสินเองไม่ได้ จะส่ง Order ไปสถานะ “รอดำเนินการด้วยตนเอง” พร้อม manual_reason บอกสาเหตุ (เช่น sku-rule, sku-unregistered, price, stock-deduct)

เปิดหลักฐาน รายงาน 2.13 ทุก Use Case

Q25 description สอดคล้องกับหน้าจออย่างไร

ตอบ

- ปุ่มและช่องกรอกในหน้าจอตรงกับคอลัมน์ User Action เช่น 5A มีปุ่ม “คุ้มค่า - สั่งซื้อเพิ่ม” และ “ไม่คุ้มค่า - ยกเลิก”
- ข้อความแจ้งเตือนบนเว็บคัดจาก description คำต่อคำ และตอนทดสอบเทียบคำต่อคำ
- description ยังเขียนชื่อ Server Action และ use-case ที่ถูกเรียกจริง เช่น `decideReorderAction` → `decideReorderUseCase`
- รายงานวางรูปหน้าจอต่อท้าย description, Sequence และ Collaboration ของ Use Case เดียวกัน ตามเกณฑ์ที่ให้เรียงทีละ Use Case

เปิดหลักฐาน รายงาน 2.13.x, 2.24

Q26 3S เลือกแหล่งสต๊อกอย่างไร ★ ทุกคนต้องตอบได้

วิธีตอบ เป็นหัวใจของ Auto Reorder ตอบเป็นลำดับขั้นตามเลข query

ตอบ

- ทำใน transaction เดียวกับ action ที่กระตุ้น **Q3S.1** ล็อกแถว stock ของ SKU (FOR UPDATE OF stock) **Q3S.2** อ่านยอด RSL ผ่าน adapter **Q3S.3** รวมจำนวนที่ถูกจองโดย Order อื่นที่ยังไม่ตัดสต๊อก แยกตาม fulfill_source
- กฎ: **RSL ก่อน** ถ้า RSL หักยอดจองแล้วพอทั้ง Order → fulfill_source = rsl ไป “ยังไม่ได้จับคู่” รอ 1S
- ถ้า RSL ไม่พอแต่**คลังบริษัทพอทั้ง Order** → fulfill_source = in_house ข้าม 1S ไป “รอจัดรูปแบบใบปะสินค้า” แล้ว 6S สร้างใบปะ INHOUSE-YAMATO-A6 ทันทีใน transaction เดียวกัน
- ถ้าไม่มีแหล่งใดพอ → “รอ Admin ตัดสินใจสั่งซื้อ” (5A) บันทึกด้วย **Q3S.4**
- ไม่แบ่งส่ง (ไม่เอา RSL ครึ่ง คลังบริษัทครึ่ง) เพราะจะได้ 2 พัสดุ 2 ใบปะ ค่าส่งเพิ่ม และ 1 Order ใช้ได้แหล่งเดียว

เปิดหลักฐาน Use Case Description 3S, Sequence 3S (สไลด์ 20–22), test/3S-inhouse-ok.png (ORD-24194)

> **ระวัง** การล็อกและหักยอดจองกันกรณี Order สองรายการแย่งของชิ้นสุดท้ายพร้อมกัน ตัวที่สองต้องรอตัวแรก commit แล้วเห็นยอดจองใหม่

Q27 6A ทำอะไรบ้าง รับสินค้าเข้าคลังอยู่ตรงไหน ★ ทุกคนต้องตอบได้

ตอบ

- ส่งคำสั่งซื้อ: Q6A.1–Q6A.5 อ่านรายการ Supplier SKU และคำสั่งซื้อที่ค้าง Q6A.6 ขอเลขใบสั่งซื้อ `SELECT nextval('reorder_no_seq')` (migration 008 เลขไม่ซ้ำแม้ลบใบล่าสุดใน 10A) Q6A.7 สร้างแถว reorders “รอส่งคำสั่งซื้อ” Q6A.8 ผูก Order ที่รอของ SKU นี้**ทุกรายการ**ด้วย `or–`

ders.reorder_id Q6A.9 เมื่อ Supplier รับ เปลี่ยนเป็น “สั่งซื้อแล้ว”

- รับสินค้า (เพิ่มในรอบนี้ ไม่ได้เพิ่ม Use Case ใหม่ เป็นขั้นที่ 4–5 ของ 6A): Q6A.10 ล็อก reorder กับ stock, Q6A.11 เปลี่ยนเป็น “ได้รับสินค้าแล้ว” พร้อม received_qty (ทำได้ครั้งเดียว), Q6A.12–Q6A.13 เพิ่ม in_house_qty และบันทึก stock_adjustment_history, Q6A.14 ส่ง Order ที่ผูกไว้กลับ “รอตรวจสอบสต๊อก” แล้ว 3S และ 6S รันต่อทันที
- ตัวอย่างทดสอบ PO-0091 รับ 60 ชิ้น ได้ “เพิ่ม OUT-CHR-003 เข้าสต๊อกในบริษัท 60 ชิ้นแล้ว” และ Order ที่ผูกไว้ 2 รายการกลับไปตรวจสต๊อก

เปิดหลักฐาน Use Case Description 6A, สไลด์ 28–30, test/6A-receive-ok.png

ระวัง ER เปลี่ยนเป็น reorders “ครอบคลุม” orders 1:N เพราะคำสั่งซื้อหนึ่งครอบคลุมได้หลาย Order ของ SKU เดียวกัน ส่วน Order หนึ่งรอคำสั่งซื้อได้ครั้งละรายการ

Q28 7A คืนสต๊อกได้อย่างไร

ตอบ

- ถ้า Order ยังไม่ส่งมอบ 7A ยกเลิกเป็น “ยกเลิกแล้ว” ถ้ามีรายการที่ตัดสต๊อกไปแล้ว คืนเข้าแหล่งเดิมตาม fulfill_source (rsl หรือ in_house) แล้วตั้ง stock_deducted กลับเป็น FALSE
- ถ้าส่งมอบให้ขนส่งแล้ว ไม่ยกเลิกทันที แต่ไปที่ “รอดำเนินการพิเศษ” ให้ประสานขนส่ง
- ถ้าเลือก “ยกเลิกคำสั่งซื้อด้วย” จะยกเลิกเฉพาะคำสั่งซื้อที่ยังเปิด และ **Q7A.5** ส่ง Order อื่นที่ผูกกับคำสั่งซื้อนั้นกลับไป “รอ Admin ตัดสินใจสั่งซื้อ” (5A)

เปิดหลักฐาน Use Case Description 7A, สไลด์ 33

Q29 ถ้า Admin เปิดสองแท็บแล้วกดอนุมัติ 5A พร้อมกันจะเกิดอะไร

ตอบ

- Q5A.1 ล็อกแถว Order ด้วย `FOR UPDATE` คำขอที่สองต้องรอคำขอแรก commit ก่อน แล้วจะเห็นว่าสถานะไม่ใช่ “รอ Admin ตัดสินใจสั่งซื้อ” แล้ว จึงได้ข้อความว่า Order ถูกตัดสินใจไปแล้ว ไม่บันทึกซ้ำ
- แบบเดียวกันใช้กับ 6A รับสินค้า สองคำขอพร้อมกันได้ ok หนึ่งครั้ง และ already หนึ่งครั้ง สต๊อกเพิ่มครั้งเดียว (ทดสอบแล้วใน code review)

เปิดหลักฐาน Use Case Description 5A ขั้นที่ 3, สไลด์ 46

5. Sequence และ Collaboration Diagram 6 ข้อ

สไลด์ 17–33 ในเล่ม 2.13

ผู้ตอบหลัก นายอชิระ ภู่รุ่งเรือง

Q30 object ที่ติดกับ actor ใน Sequence Diagram คืออะไร ★ ทุกคนต้องตอบได้

ตอบ

- เป็นหน้าจอ (page ของ Next.js) ชื่อ `:<X>Screen` เช่น `:OrderVerifyScreen` คือ**ชั้น View ของ MVC** Actor กดที่หน้าจอนี้เท่านั้น
- ทำไมชื่อ Screen ไม่ใช่ View: เกณฑ์ข้อ 24 เขียนว่า object ติด actor ต้องเป็น user interface และ**ห้ามใช้ชื่อว่า view** จึงตั้งชื่อว่า Screen แต่หน้าที่ยังเป็น View ของ MVC ตามที่อาจารย์สอน
- ลำดับทุกรูป: Super Admin → Screen (หน้าจอ) → Controller (Server Action ใน `actions.ts`) → Model (UseCase / ExecuteWorkflow + Queries) → `:DBConnector` และ Adapter สำหรับระบบภายนอก
- ทุกเส้นที่ต่อฐานข้อมูลผ่าน DBConnector มีเลข query กำกับ `check_diagrams.py` ตรวจว่าเลข query ทุกข้อใน description อยู่ใน Sequence และ Collaboration ครบ และตรวจว่าไม่มีลูกศรกลับหา Actor

เปิดหลักฐาน สไลด์ 17 (ตาราง MVC) และ 21, 26, ไฟล์ mmd/<UC>-sequence.mmd

> **ระวัง** ทุก รูป ใน เล่ม สไลด์ และ Class Diagram ใช้ ชื่อ Screen ไม่มี object ชื่อ View `check_diagrams.py` ตรวจว่าไม่มีชื่อ object ที่มีคำว่า View ตอบสั้น ๆ ว่า “Screen คือชั้น View ของ MVC ตั้งชื่อตามเกณฑ์ข้อ 24”

Q31 โค้ดใช้ Server Action ไม่มีคลาส Controller ทำไมใน Sequence มี Screen และ Controller ★ ทุกคนต้องตอบได้

ตอบ

- Screen (ชั้น View) กับ Controller เป็น**ชื่อบทบาท (stereotype)** ตามแบบ MVC ที่อาจารย์สอน จับคู่กับไฟล์จริงหนึ่งต่อหนึ่ง ไม่ได้สร้างคลาสใหม่ในโค้ด
- Screen = `src/app/(app)/.../page.tsx` รับคลิกและแสดงผล
- Controller = Server Action ใน `src/app/(app)/actions.ts` เช่น `verifyOrderAction` ตรวจ session รับค่าจากหน้าจอ แล้วเรียก use-case Next.js ส่งคำขอจากหน้าจอมาที่ฟังก์ชันนี้ ทำหน้าที่เดียวกับ Controller
- Model = `src/lib/use-cases/*` (ลำดับขั้นของ UC) + `src/lib/queries/*` (SQL ค่าคงที่ `Q<UC>_<n>`) และ `db.ts` คือ DBConnector
- วงเล็บใต้ชื่อ object ในรูปบอกชื่อไฟล์จริง เช่น `(actions.ts)` เปิดโค้ดเทียบได้ทันที

เปิดหลักฐาน สไลด์ 17, สไลด์ 44 (Layers), รายงาน 2.18 Class Diagram

วิธีตอบ ห้ามพูดว่าโค้ดมีคลาส Controller หรือ Repository ให้พูดว่า “Controller ในรูปคือ Server Action”

Q32 ทำไมไม่มีลูกศรส่งกลับไปที่ Actor ★ ทุกคนต้องตอบได้

ตอบ

- Actor เป็นคน ไม่ใช่ object ในระบบ ระบบส่ง message ให้คนไม่ได้ สิ่งที่คนเห็นคือหน้าจอเปลี่ยน
- ผลลัพธ์จึงย้อนกลับเป็นเส้นประ (return) จาก DBConnector → Model → Controller → Screen แล้ว Screen แสดงผลเองเป็น **self message** เช่น `showSuccess()`, `showError(...)`
- ตามที่อาจารย์สอน และ `check_diagrams.py` ตรวจทุก Sequence ว่าไม่มีเส้นที่ปลายทางเป็น Actor
- Use Case ที่ระบบทำเอง (S) ไม่มีหน้าจอของตัวเอง ผลขึ้นเป็นข้อความแจ้งเตือนในหน้าที่ Admin เปิดอยู่ ก็เป็น self message บน Screen เช่นกัน

เปิดหลักฐาน สไลด์ 17 บรรทัด `Screen ->> Screen : showSuccess()`, สไลด์ 21

Q33 Sequence กับ Collaboration ต่างกันอย่างไร

ตอบ

- Sequence เน้นลำดับเวลา อ่านจากบนลงล่าง
- Collaboration แปลงจาก Sequence ใช้ object ชุดเดียวกัน แต่จัดตามความสัมพันธ์ของ object และใส่เลขลำดับบนทุกข้อความแทนแกนเวลา กรณีแยกทางเขียนเงื่อนไขในวงเล็บเหลี่ยม
- สร้างด้วยสคริปต์ `gen_collab.py` จาก mmd ของ Sequence จึงไม่หลุดกัน แล้ว render ด้วย Graphviz

เปิดหลักฐาน สไลด์ 21–22 และ 29–30

Q34 DBConnector ใน diagram โปรแกรมใช้จริงไหม ★ ทุกคนต้องตอบได้

ตอบ

- ใช้จริงครับ/ค่ะ DBConnector คือ `src/lib/db.ts` สร้าง connection pool ของ node-postgres (pg) ตัวเดียว
- มีฟังก์ชัน `withTransaction` เปิด BEGIN บน connection เดียว แล้ว COMMIT หรือ ROLLBACK ทุก action ที่เขียนข้อมูลผ่านตัวนี้
- ทุกหน้าในเว็บอ่านเขียน PostgreSQL จริง ไม่เก็บข้อมูลใน localStorage แล้ว (เหลือแค่จำภาษาที่เลือก) ข้อมูลตั้งต้นสร้างด้วย `npm run db:reset` จาก migrations 001–008

เปิดหลักฐาน 01418321-final-project/src/lib/db.ts, รายงาน 2.20, สไลด์ 44

Q35 ทำไมแยกชั้น ไม่ให้หน้าจอเรียกฐานข้อมูลเลย

ตอบ

- Server Action (`actions.ts`) ตรวจ session และรับค่าจากหน้าจอ use-case คุมลำดับขั้นของ Use Case เช่น เช็กสถานะก่อนบันทึก ส่วน queries รู้แค่ SQL ตามเลข Q
- Use Case ที่ระบบทำเอง (2S, 3S, 6S, 7S) ไม่มีหน้าจอ เรียก use-case ตัวเดียวกันได้จาก `execute–Workflow`
- ระบบภายนอกแยกอยู่หลัง adapter ตอนเปลี่ยนจาก mock เป็นของจริงไม่ต้องแก้ use-case

เปิดหลักฐาน Class Diagram 2.18, Component Diagram, สไลด์ 44

ระวัง พูดคำว่า Controller ได้ในความหมายบทบาท (= Server Action) แต่ห้ามบอกว่าโค้ดมีคลาส Controller หรือ Repository (Q31)

6. ER Diagram และโครงสร้างตาราง

6 ข้อ

สไลด์ 34–36 ในเล่ม 2.14–2.17

ผู้ตอบหลัก นางสาวปิยากร ผลพานิช

Q36 ER มี M:N ไหม total participation ดูตรงไหน ★ ทุกคนต้องตอบได้

ตอบ

- ER แบบ Chen มี **17 entity** และ **22 relationship** เป็น 1:1 อยู่ 2 คู่ (skus–stock และ stock–rakuten_stock_sync_states) ที่เหลือ 1:N ทั้งหมด **ไม่มี M:N** จึงไม่ต้องมีรูป logical schema เพิ่ม
- เส้นคู่คือ total participation (FK เป็น NOT NULL) เช่น order_items ทุกแถวต้องมี Order ส่วนเส้นเดี่ยวคือ partial เช่น Order ไม่จำเป็นต้องผูกกับคำสั่งซื้อเติมสต๊อก
- attribute ที่ขีดเส้นใต้คือ primary key ตัวเอียงคือ foreign key

เปิดหลักฐาน รูป 2.14, สไลด์ 34

Q37 ทำไมเดิมบอก 15 ตาราง ตอนนี้เป็น 17 ★ ทุกคนต้องตอบได้

วิธีตอบ ตอบว่าแบบแรกเป็นแบบวิเคราะห์ แล้วเราปรับให้ตรงฐานข้อมูลจริงของเว็บ ไม่ต้องอาย เป็นหลักฐานว่าแบบกับโค้ดตรงกัน

ตอบ

- 17 ตารางตรงกับ migrations 001–008 ของเว็บ ไม่นับ `schema_migrations` ที่ตัวรัน migration ใช้จดว่ารันไฟล์ไหนแล้ว
- **เปลี่ยนชื่อ**: users → admins, products → skus, purchase_orders → reorders, label → labels, deletion_log → deletion_history, product_cost + cost_calculation_log → costs + cost_history
- **แยก**: orders แยกเป็น orders + order_items (แก้ปัญหา Order กับสินค้าเป็น M:N)
- **ตัดออก**: sku_rules (ย้ายเป็นคอลัมน์ skus.channel_sku), shipping_rules (กฎใบปะอยู่ในโค้ด), rsl_shipments, delivery_partners, carrier_tracking (ข้อมูลมาจาก adapter ไม่ต้องเก็บ)
- **เพิ่ม**: sessions, auto_error_logs, tracking_notification_logs, rakuten_stock_sync_states, stock_adjustment_history, label_print_history

เปิดหลักฐาน รายงาน 2.14 และ 2.17, sql/schema.sql

Q38 Order หนึ่งมีสินค้าหลายชนิดได้ไหม Order กับสินค้าเป็น M:N หรือเปล่า

ตอบ

- ในแบบมีตาราง `order_items` เป็นตัวกลาง orders 1:N order_items และ skus 1:N order_items จึงไม่มี M:N ตรง ๆ

- แต่ละรายการเก็บ channel_sku, qty, fulfill_source และ stock_deducted ของตัวเอง การตัดสต๊อก (7S) และคืนสต๊อก (7A) ทำทีละรายการ

ระวัง query ที่แสดงผลหลายข้อ (เช่น Q2A.1, Q6S.1) อ่านรายการแรกด้วย LIMIT 1 และข้อมูลตัวอย่างทุก Order มี 1 รายการ ถ้าถูกถามว่ารองรับหลายรายการเต็มที่ไหม ให้ตอบว่าตารางรองรับแล้ว ส่วนหน้าจอและ 3S ยังทำงานแบบ 1 รายการต่อ Order ในเฟส 1

เปิดหลักฐาน sql/schema.sql ตาราง order_items

Q39 ทำไม ER แบ่งเป็นหลายรูป ER พร้อม Query อ่านอย่างไร

ตอบ

- 17 ตาราง **159 attribute** รวมในรูปเดียวอ่านไม่ออกบน A4 จึงมีรูปภาพรวม แล้วแยกรายละเอียดเป็น 4 ส่วน
- ER พร้อม Query แสดงเฉพาะ entity พร้อม PK และ FK ป้ายสีแดงบอกเลข query ที่กระทำกับ entity นั้น query ที่เป็น UPDATE แสดง attribute ที่ถูกแก้ด้วย ครบทั้ง 113 query

เปิดหลักฐาน รูป 2.14–2.15, สไลด์ 34–35

Q40 ฐานข้อมูล normalize ถึงขั้นไหน มีข้อมูลซ้ำไหม

ตอบ

- ตารางหลักอยู่ในรูป 3NF ทุก attribute ขึ้นกับ primary key ของตัวเอง
- ที่ตั้งใจเก็บซ้ำคือ labels เก็บ label_template และ rsl_no เป็นภาพ ณ ตอนสร้างใบปะ ถ้ากฎใบปะเปลี่ยนภายหลัง ใบที่พิมพ์ไปแล้วต้องไม่เปลี่ยนตาม และ deletion_history เก็บ snapshot แบบ JSON ของ Order ที่ลบ
- cost_history เก็บค่าก่อนและหลังเป็น JSON เพื่อเป็นประวัติ ไม่ได้ใช้แทนข้อมูลปัจจุบันใน costs

เปิดหลักฐาน sql/schema.sql

Q41 Table Structure มีอะไรบ้าง

ตอบ

- **17 ตาราง** รวม **159 attribute** แต่ละ attribute มีชื่อ คำอธิบาย PK/FK, Not Null, ชนิดข้อมูลและขนาด และช่วงค่าหรือการตรวจสอบ
- ช่วงค่าเขียนเป็น CHECK จริงในฐานข้อมูล เช่น order_status ต้องเป็น 1 ใน 22 ค่า, reorder_status 4 ค่า, received_qty $>$ 0
- ทุกตารางมีตัวอย่างข้อมูล 5 แถว ยกเว้น admins มี 2 แถว เพราะมี Super Admin คนเดียว ดึงจากข้อมูลตั้งต้นของเว็บ (`npm run db:reset`) ตารางอื่นที่มีไม่ถึง 5 แถว (เช่น sessions) เติมแถวตัวอย่างเฉพาะในเล่ม (`sql/sample_extra.sql`) ผล SQL ใน 2.23 ยังใช้ข้อมูลตั้งต้นเดิม

เปิดหลักฐาน รายงาน 2.17, sql/dictionary.json, สไลด์ 36

7. Class และ State Diagram

4 ข้อ

สไลด์ 37–43 ในเล่ม 2.18–2.19

ผู้ตอบหลัก นางสาวปิยากร ผลพานิช

Q42 Class Diagram ได้มาจากไหน มีความสัมพันธ์แบบไหนบ้าง ★ ทุกคนต้องตอบได้

ตอบ

- นำ object ทุกตัวใน Sequence และ Collaboration ของทั้ง 17 Use Case (56 participant) มาทำเป็นคลาส ตรงกับชั้นในโค้ดแบบ MVC คือ Screen (หน้าจอ ชั้น View), Controller (Server Action), UseCase (รวม ExecuteWorkflow), Queries, DBConnector, Adapter และคลาสข้อมูล (Entity) ที่ตรงกับตาราง
- ความสัมพันธ์ที่ใช้
  - association หน้าจอ (Screen) เรียก Controller
  - dependency Controller เรียก UseCase และ UseCase เรียก Queries กับ Adapter
  - aggregation Queries ทุกโมดูลใช้ pool ของ DBConnector ตัวเดียวกัน
  - realization MockAdapters ใน mock.ts implement interface ของ Adapter ทุกตัว (เทียบได้กับ super/sub)
- ทุกคลาสมี attribute และ method

เปิดหลักฐาน รายงาน 2.18, สไลด์ 37

Q43 ทำไม Class Diagram มี 15 รูป และทำไม DBConnector เป็น aggregation

ตอบ

- 12 รูปแบ่งตามกลุ่มงาน กลุ่มละไม่เกิน 2 Use Case ให้ตัวอักษรอ่านได้ อีก 3 รูปเป็นคลาสข้อมูล
- DBConnector มีอยู่ได้เองและใช้ร่วมกันทุกโมดูล queries ถ้าโมดูลหนึ่งหายไป pool ไม่ได้หายตาม จึงเป็น aggregation ไม่ใช่ composition

เปิดหลักฐาน รายงาน 2.18 (Class Diagram 1/15 ถึง 15/15)

Q44 State Diagram บอก SQL อย่างไร ★ ทุกคนต้องตอบได้

ตอบ

- มี State Diagram ทุกคอลัมน์สถานะที่ถูก update (Q45) รูปหลักคือ Order.order_status เขียนแบบ class.state ทุกเส้นเขียนแบบ [เงื่อนไข] event / Use Case และกำกับเลข query ที่ UPDATE order_status
- ครบทั้ง **22 สถานะ** ใน order-status.ts หลัง 3S มี 3 ทาง [RSL พอ] → ยังไม่ได้จับคู่, [RSL ไม่พอ คลังบริษัทพอ] → รอจัดรูปแบบใบปะสินค้า, [ไม่มีแหล่งใดพอ] → รอ Admin ตัดสินใจสั่งซื้อ

- “สั่งซื้อแล้ว” ออกได้ 2 ทาง รับสินค้า (Q6A.14) กลับไป “รอตรวจสอบสต๊อก” หรือ 7A ยกเลิกคำสั่งซื้อ (Q7A.5) กลับไป 5A
- “จัดส่งสำเร็จ” ได้เมื่อครบ 3 เงื่อนไข ขนส่งแจ้ง delivered, แจ้งเลขติดตามแล้ว และตัดสต๊อกครบทุกรายการ ตรวจที่ Q7S.7, Q5S.4 และ Q8A.4/Q8A.5

เปิดหลักฐาน รายงาน 2.19, สไลด์ 38

Q45 มี State Diagram ของตารางอื่นไหม ข้อมูลไหนบ้างที่ถูก update ★ ทุกคนต้องตอบได้

ตอบ

- มีครบทุกคอลัมน์สถานะที่โค้ด UPDATE รวม 5 คลาส 6 รูป (Order แบ่ง 2 รูป) ทุกเส้นเขียน [เงื่อนไข] event / Use Case (Query)
- `Order.order_status` 22 สถานะ (สไลด์ 38–40)
- `Reorder.reorder_status` รอส่งคำสั่งซื้อ (Q6A.7) → สั่งซื้อแล้ว (Q6A.9) → ได้รับสินค้าแล้ว (Q6A.11) หรือยกเลิกแล้ว (Q7A.4) ลบได้ใน 10A (Q10A.4) (สไลด์ 41)
- `Label.label_status` ready (6S Q6S.4) → printed (9A Q9A.4) ยกเลิกจับคู่ RSL ลบใบปะที่ยังไม่พิมพ์ (Q1S.4) (สไลด์ 42)
- `Sku.sku_status` active ↔ inactive ใน 3A (Q3A.9) ลบเมื่อไม่มีข้อมูลอ้างถึง (Q3A.11) (สไลด์ 42)
- `Order.delivery_status` in_transit (8A Q8A.2) → delivered / failed / returned (Q8A.4) (สไลด์ 43)
- `RakutenStockSyncState.sync_status` pending เมื่อยอดสต๊อกเปลี่ยน (3S, 6A, 7S) → synced เมื่อ Rakuten รับยอด (Q7S.10) ส่งไม่สำเร็จคง pending (Q7S.11) (สไลด์ 43)
- ตารางประวัติ (เช่น cost_history, label_print_history) INSERT อย่างเดียวไม่มีสถานะ จึงไม่มี State Diagram

เปิด หลัก ฐาน รายงาน 2.19, `gen_diagrams.py` ส่วน OTHER_STATES, sql/ schema.sql ช่อง CHECK

8. สถาปัตยกรรมเว็บจริง

7 ข้อ

สไลด์ 44–48 ในเล่ม 2.18, 2.20

ผู้ตอบหลัก นางสาวปิยากร ผลพานิช

Q46 เว็บแบ่งชั้นอย่างไร มี REST API ไหม ★ ทุกคนต้องตอบได้

ตอบ

- หน้าจอ (page ของ Next.js App Router) → Server Action ใน `src/app/(app)/actions.ts` → use-case ใน `src/lib/use-cases` → queries ใน `src/lib/queries` (SQL เป็นค่าคงที่ `Q<UC>_<n>`) → `src/lib/db.ts`
- ระบบภายนอกเรียกผ่าน `src/lib/adapters` ไม่มี REST API แยก เพราะหน้าจอกับ server อยู่ในโปรเจกต์เดียว Server Action เป็นทางเรียกฝั่ง server
- ไม่มีคลาส Controller, Repository หรือ Scheduler

เปิดหลักฐาน สไลด์ 44, Component Diagram

Q47 ทำไมไม่มี Scheduler แล้ว Use Case อัตโนมัติทำงานตอนไหน ★ ทุกคนต้องตอบได้

ตอบ

- ทุก action ของ Admin ที่ทำให้ Order เปลี่ยนสถานะ เรียกผ่าน `executeWorkflow` ซึ่งรัน action แล้วรัน `runAutomation` ต่อ**ใน transaction เดียวกัน** เช่น 2A ยืนยัน → 2S → 3S → 6S ถ้าเป็นคลังบริษัท
- `runAutomation` มีรอบ retry ทุกครั้ง หยิบ Order ที่ค้าง “รอจับคู่กฎ SKU” (2S) และ Order ที่ “แจ้งเลขติดตามแล้ว” แต่ตัดสต๊อกไม่ครบ (Q7S.1) มาทำต่อ แล้วซิงก์ยอดสต๊อกไป Rakuten
- เปิดหน้า `/shipping` จะดึงสถานะขนส่ง (8A) และเปิด `/shipping/label` จะทำ 6S ให้ Order ที่รอ
- เหตุผล: ระบบมีผู้ใช้คนเดียว Order เข้ามาเมื่อ Admin กดนำเข้า ทำทันทีให้เห็นผลเลย ไม่ต้องดูแล process แยก ถ้าส่วนอัตโนมัติพลาด ทั้ง action rollback ข้อมูลไม่ค้างครึ่งทาง
- ระบบจริงที่ดึง Order ตามรอบเวลา จะเพิ่ม cron เรียก use-case เดิมได้ เป็นงานเฟสถัดไป

เปิดหลักฐาน `src/lib/use-cases/execute-workflow.ts`, รายงาน 2.11, สไลด์ 45

Q48 ถ้าส่วนอัตโนมัติพลาดกลางทางจะเกิดอะไร

ตอบ

- ข้อผิดพลาดของระบบภายนอกที่ description กำหนดไว้ (เช่น RSL ล่ม) ไม่ทำให้ rollback ระบบตั้ง Order ไปสถานะรอหรือ “รอดำเนินการด้วยตนเอง” ตามทางเลือก
- ถ้าเป็นข้อผิดพลาดจริงของฐานข้อมูลในขั้นอัตโนมัติ ทั้ง transaction rollback รวมถึง action ที่กดแล้วบันทึกลง `auto_error_logs` ใน transaction ใหม่ (เช่น Q3S.5, Q7S.12) ผู้ใช้เห็นข้อความแจ้ง

เปิดหลักฐาน `execute-workflow.ts` ฟังก์ชัน recordRolledBackAutomationError

Q49 กันตัดสต๊อกซ้ำอย่างไร ★ ทุกคนต้องตอบได้

ตอบ

- **Q7S.2** ล็อกรายการสินค้าและแถว stock (`FOR UPDATE OF item, stock`) ก่อนตัด และเลือกเฉพาะรายการที่ `stock_deducted = FALSE`
- **Q7S.4** ลดยอดตามแหล่ง fulfill_source แล้ว **Q7S.5** ตั้ง `stock_deducted = TRUE` โดยมีเงื่อนไข `AND stock_deducted = FALSE` ซ้ำอีกชั้น ถ้ามีคำขอที่สองเข้ามาจะไม่เหลือรายการให้ตัด
- รอบ retry **Q7S.1** ใช้ `FOR UPDATE SKIP LOCKED` ไม่เกิน 100 รายการ จึงไม่แย่ง Order ที่ action อื่นกำลังทำ และข้าม Order ที่ action นี้ขอไว้แล้ว
- ถ้าสต๊อกไม่พอ Order ไป “รอดำเนินการด้วยตนเอง” (stock-deduct) ด้วย Q7S.3 จึงไม่วน retry ไม่จบ
- 7A คืนสต๊อกเฉพาะรายการที่ stock_deducted = TRUE คืนเข้าแหล่งเดิม แล้วตั้งกลับเป็น FALSE

เปิดหลักฐาน Use Case Description 7S, สไลด์ 32 และ 46

Q50 ใช้ transaction และ row lock ที่ไหนบ้าง

ตอบ

- ทุก action ที่เขียนข้อมูลอยู่ใน `withTransaction` (BEGIN/COMMIT/ROLLBACK บน connection เดียว)
- row lock ที่สำคัญ Q3S.1 ล็อก stock ก่อนเลือกแหล่ง, Q5A.1 ล็อก Order ก่อนตัดสินใจ, Q6A.10 ล็อก reorder กับ stock ก่อนรับสินค้า, Q7S.2 ล็อกรายการกับ stock ก่อนตัด, Q2A.2/Q1S.2/Q6S.1/Q7A.1 ล็อก Order ก่อนเปลี่ยนสถานะ
- ข้อจำกัดที่รู้: รับสินค้าล็อก reorder ก่อน Order ส่วน 7A ล็อก Order ก่อน reorder อาจ deadlock ได้ถ้ากดพร้อมกันพอดี PostgreSQL จะยกเลิกตัวหนึ่ง action นั้นล้มแบบสะอาด ผู้ใช้กดใหม่ได้ ระบบมีผู้ใช้คนเดียวจึงโอกาสต่ำ

เปิดหลักฐาน สไลด์ 46, รายงาน 2.13 (3S, 5A, 6A, 7S)

Q51 adapter จำลองคืออะไร จะต่อของจริงต้องทำอะไร ★ ทุกคนต้องตอบได้

ตอบ

- `src/lib/adapters/contracts.ts` กำหนด interface 6 ตัว คือ RakutenAdapter, RslAdapter, LabelServiceAdapter, PrinterAdapter, CarrierAdapter, SupplierAdapter use-case เรียกผ่าน interface เท่านั้น
- ตอนนี้ทุกตัวเป็น mock (`mock.ts`) โหมดที่ตั้งได้มีแค่ `"mock"` ผ่านตัวแปร เช่น `RAKUTEN_ADAPTER` ข้อมูลจำลองอยู่ใน `src/mock`
- ข้อผิดพลาดของระบบภายนอกจำลองที่หน้า `/dev` (ไม่อยู่ในเมนู) มี **11 สวิตช์** เช่น RMS ล่ม, RSL ล่ม,

เครื่องพิมพ์ไม่พร้อม, ฐานข้อมูลบันทึกไม่ได้

- ต่อของจริง: เขียน adapter ใหม่ที่ implement interface เดิม (เช่นเรียก Rakuten RMS API ด้วย API key ของร้าน) เพิ่มโหมดใน `adapters/index.ts` แล้วตั้งค่าตัวแปร ไม่ต้องแก้ use-case, SQL หรือหน้าจอ

เปิดหลักฐาน `src/lib/adapters`, สไลด์ 47

Q52 ระบบปลอดภัยแค่ไหน เก็บรหัสผ่านและ session อย่างไร ★ ทุกคนต้องตอบได้

ตอบ

- **รหัสผ่าน** เก็บเป็น scrypt (N = 16384, r = 8, p = 1, key 64 ไบต์, salt สุ่ม 16 ไบต์ต่อบัญชี) ไม่เก็บตัวอักษรตรง ๆ เทียบด้วย `timingSafeEqual` กันการวัดเวลา
- **session** token สุ่ม 32 ไบต์ ในฐานข้อมูลเก็บแค่ SHA-256 ของ token (`sessions.token_hash`) ถ้าฐานข้อมูลหลุดก็เอา token ไปใช้ไม่ได้ cookie เป็น httpOnly, sameSite = lax, secure ใน production อายุ 8 ชั่วโมง ออกจากระบบตั้ง revoked_at (Q1A.7)
- **ล็อกบัญชี** ผิดครบ **5 ครั้ง** ล็อก **15 นาที** (Q1A.3) ครบเวลาปลดเอง (Q1A.2) เข้าสำเร็จล้างตัวนับ (Q1A.4) ระหว่างล็อกไม่ตรวจรหัสผ่าน
- **SQL injection** ทุก query ใช้ parameter $1, $2 ไม่ต่อ string

ระวัง จุดอ่อนที่รู้: ผิดครบ 5 ครั้งข้อความเปลี่ยนเป็น “บัญชีถูกล็อก” จึงบอกได้ว่าอีเมลนั้นมีอยู่ อีเมลที่ไม่มีในระบบไม่ถูกนับและไม่ถูกล็อก ระบบไม่มี 2FA (นอกขอบเขตเฟส 1)

เปิดหลักฐาน `src/lib/auth/password-core.ts`, `sessions.ts`, Use Case Description 1A, สไลด์ 18 และ 48

9. Platform, Site map, UI, รายงาน

5 ข้อ

สไลด์ 49–51 ในเล่ม 2.20–2.22

ผู้ตอบหลัก นางสาวปิยากร ผลพานิช

Q53 ใช้เครื่องมืออะไรพัฒนา ทำไมเลือก

ตอบ

- Next.js 16 (App Router) กับ React 19 และ TypeScript 5 ทำทั้งหน้าจอและฝั่ง server ในโปรเจกต์เดียว ทีมเดียวดูแลได้
- Tailwind CSS 4 กับ shadcn/ui ทำหน้าจอ TanStack Table ทำตาราง Recharts ทำกราฟ Sonner แสดงข้อความแจ้งเตือน
- PostgreSQL ต่อผ่าน node-postgres (pg) รัน SQL ทดสอบด้วย PGlite
- draw.io วาด BP และ Use Case, Mermaid กับ Graphviz วาด Sequence, Collaboration, ER, Class, State

เปิดหลักฐาน รูป Stack of Tools ใน 2.20, สไลด์ 49

Q54 Component Diagram อ่านอย่างไร

ตอบ

- Screens (หน้าจอ Next.js ชั้น View) → Controllers (Server Actions ใน actions.ts) → UseCase และ ExecuteWorkflow → Queries → DBConnector → PostgreSQL 17 ตาราง
- UseCase ต่อระบบภายนอกผ่าน Adapters (mock) คือ Rakuten RMS, RSL, Label Service, เครื่องพิมพ์, ขนส่ง และ Supplier
- เป็นโครงของเว็บที่ทำงานจริงตอนนี้ ไม่ใช่แบบที่วางไว้เฉย ๆ

เปิดหลักฐาน สไลด์ 49, diagrams/png/component.png

Q55 Site map มีโครงสร้างอย่างไร ชื่อหน้าจอมาจากไหน

ตอบ

- ผู้ใช้ทุกคนเข้าผ่านหน้าเดียวกัน จึงมี tree เดียว root คือ LoginScreen ตามเกณฑ์
- หลังเข้าสู่ระบบไป DashboardScreen ซึ่งมีปุ่มนำเข้า Order (4A) เมนูด้านซ้ายแบ่ง 5 กลุ่ม รวม **12 หน้าจอ** บวกหน้าต่างรับสินค้าของ ReorderScreen
- ชื่อหน้าจอตรงกับ object Screen ใน Sequence เช่น หน้าจอ ReorderScreen = :ReorderScreen บางหน้าจอรับหลาย Use Case เช่น ReorderScreen ทำ 5A และ 6A, ShippingScreen ทำ 8A และปุ่มส่งเลขติดตามของ 5S
- 2S, 6S, 7S ไม่มีหน้าจอของตัวเอง ผลขึ้นเป็นข้อความแจ้งเตือนในหน้าที่ Admin เปิดอยู่

เปิดหลักฐาน รายงาน 2.21, สไลด์ 50, หน้า /sitemap ในเว็บ

Q56 ระบบมีรายงานอะไรบ้าง ★ ทุกคนต้องตอบได้

ตอบ

มี **2 รายงาน**

- ใบปะสินค้า สร้างจาก 6S พิมพ์จาก 9A มี 3 template ตามแหล่งจัดส่ง RSL-STD-A6, RSL-LARGE-A5 และ INHOUSE-YAMATO-A6 (บริษัทส่งเอง ช่อง RSL เป็น “—” และเลขสแกนคือ Order ID กับเลขกล่อง) ตัวอย่าง ORD-24178 (RSL) และ ORD-24177 (in-house)
- รายงานต้นทุนต่อหน่วยจาก 4S แสดงองค์ประกอบต้นทุนแยกบรรทัด ต้นทุนต่อหน่วยเทียบราคาขาย และประวัติจาก cost_history ตัวอย่าง FSH-MLR-240

เปิดหลักฐาน รายงาน 2.22, สไลด์ 51

Q57 ต้นทุนต่อหน่วยคำนวณอย่างไร ★ ทุกคนต้องตอบได้

ตอบ

- สูตร ((purchase_price × exchange_rate) + intl_freight + duty_fee) / order_qty + market-place_fee + domestic_shipping + rsl_charge
- ส่วนแรกเป็นค่าทั้งล็อตหารจำนวนที่สั่ง ส่วนหลังเป็นค่าต่อชิ้นอยู่แล้ว
- ตัวอย่าง FSH-MLR-240 คือ 9,100 × 4.92 = 44,772 บวก 8,600 บวก 3,150 ได้ 56,522 หาร 200 ได้ 282.61 บวก 72 + 46 + 18 ได้ **418.61 บาท** ราคาขาย 1,280 บาท
- สูตรคำนวณใน use-case (ไม่ใช่ SQL) แล้วบันทึกผลด้วย Q4S.5 (costs) และประวัติด้วย Q4S.6 (cost_history) 5A อ่านค่าล่าสุดใน Q5A.1

เปิดหลักฐาน Use Case Description 4S, รายงาน 2.22, สไลด์ 24

ระวัง ข้อมูลตัวอย่างใช้อัตรา 4.92 กับสกุล CNY ซึ่งเป็นข้อมูลสมมติ สูตรไม่ขึ้นกับค่า อัตราจริงเจ้าของกรอกเอง

10. SQL

3 ข้อ

สไลด์ 36 และ 52 ในเล่ม 2.16, 2.23

ผู้ตอบหลัก นางสาวปิยากร ผลพานิช

Q58 SQL รันจริงหรือเปล่า ผลดูตรงไหน ★ ทุกคนต้องตอบได้

ตอบ

- รันจริงสองทาง ทางแรก `npm run sql` ในโฟลเดอร์เครื่องมือของเล่ม `04-chapter2-diagrams-database` (ไม่ได้อยู่ใน repo เว็บ) สร้างฐานข้อมูล 17 ตารางจาก schema.sql กับ seed.sql ใน PGlite (PostgreSQL ตัวจริงที่รันใน Node.js) แล้วรันครบ **113 คำสั่ง** ผ่านทุกข้อ (0 failed)
- แต่ละข้อรันแยกจากข้อมูลตั้งต้นชุดเดียวกัน คำสั่งที่แก้ข้อมูลแสดงข้อมูลหลังรันด้วย
- ทางที่สอง SQL ในเล่มเป็นข้อความเดียวกับค่าคงที่ใน `src/lib/queries` ของเว็บ (แต่ละข้อจดชื่อค่าคงที่คู่กันไว้ใน qmap ของ usecases.json เทียบทีละข้อแล้วตรงกัน) จึงเป็นคำสั่งที่เว็บใช้งานอยู่จริง

เปิดหลักฐาน รายงาน 2.23, สไลด์ 52, sql/sql-results.json

Q59 SQL พวกนี้ใช้ AI เขียนหรือเปล่า ทำไมบางข้อยาว

วิธีตอบ เกณฑ์ข้อ 33 เขียนว่าไม่ควรใช้ AI generate ตอบตามความจริงของทีม ห้ามแต่ง สิ่งที่ช่วยได้คือพิสูจน์ว่าเราอธิบายได้ทุกข้อ ให้อาจารย์สุ่มเลข query มาแล้วอธิบายเลย

ตอบ

- SQL เขียนตามขั้นตอนใน description ทีละขั้น ข้อหนึ่งทำงานเดียว
- ข้อที่ยาว เช่น Q2A.1 เป็น query อ่านรวมของหน้าจอ และข้อที่ขึ้นต้นด้วย WITH ใช้แก้สองตารางพร้อมกันในคำสั่งเดียว เช่นปรับสต๊อกพร้อมบันทึกประวัติ
- รวม 113 ข้อ เป็น SELECT 50, UPDATE 30, WITH 16, INSERT 15, DELETE 2

เปิดหลักฐาน Query Table 2.16

Q60 10A ลบตามลำดับไหน ทำไม

ตอบ

- Q10A.1 ลบ Order ก่อน เก็บ snapshot ลง deletion_history แล้วลบ Order order_items กับ labels ถูกลบตาม ON DELETE CASCADE
- Q10A.2 ลบใบปะตามช่วงวันที่ Q10A.3 ลบประวัติต้นทุน Q10A.4 ลบคำสั่งซื้อเติมสต๊อกเฉพาะที่ไม่มี Order ที่ยังเปิดอ้างถึงผ่าน orders.reorder_id Q10A.5 อ่านประวัติการลบ
- ทุกข้อตรวจเงื่อนไขซ้ำใน SQL (เช่นสถานะต้องปิดแล้ว) และทำใน transaction เดียว ไม่เกิน 1,000 รายการต่อครั้ง

เปิดหลักฐาน Use Case Description 10A

11. การทดสอบและเดโม

5 ข้อ

สไลด์ 53–54 ในเล่ม 2.24

ผู้ตอบหลัก นางสาวปิยากร ผลพานิช

Q61 ตอนนี้เว็บต่อฐานข้อมูลจริงหรือยัง ★ ทุกคนต้องตอบได้

ตอบ

- ต่อแล้วครับ/ค่ะ ทุกหน้าอ่านเขียน PostgreSQL ผ่าน Server Action → use-case → queries → db.ts ไม่เก็บข้อมูลใน localStorage แล้ว (เหลือแค่จำภาษาที่เลือก)
- schema มาจาก migrations 001–008 ข้อมูลตั้งต้นสร้างด้วย `npm run db:reset` ที่ใช้ตอนทดสอบและเดโม
- ส่วนที่ยังจำลอง คือ ระบบภายนอก (Rakuten RMS, RSL, ขนส่ง, เครื่องพิมพ์, Supplier) ผ่าน adapter (Q51)

เปิดหลักฐาน รายงาน 2.20, สไลด์ 44

Q62 ทดสอบอะไรไปแล้วบ้าง ครอบคลุมแค่ไหน ★ ทุกคนต้องตอบได้

ตอบ

- **ทดสอบตาม Use Case 36 กรณี** บนเว็บจริง ทุก Use Case มีกรณีปกติและผิดพลาด บวก 3S in-house (ORD-24194) และ 6A รับสินค้า (PO-0091) ผ่านทุกกรณี ทุกกรณีมีรูปหน้าจอ
- **ระบบภายนอกล่ม** เปิดสวิตช์ในหน้า `/dev` แล้วดูว่าข้อความตรงกับ description คำต่อคำ และสถานะ Order คงเดิมตามที่ description บอก
- **SQL** 113 ข้อ รันด้วย `npm run sql` ในโฟลเดอร์เครื่องมือของเล่ม `04-chapter2-diagrams-database` ผ่านทุกข้อ
- **โค้ด** `npm run lint`, `npx tsc --noEmit`, `npm run build` ผ่าน และ `npm run test:phase8-reset` ทดสอบ migrate/seed/reset บน schema ชั่วคราว (ตรวจครบ 17 ตาราง)
- ระหว่างแก้โค้ดมีสคริปต์เดินทั้ง flow (RSL, in-house, สั่งซื้อและรับของ, ยกเลิก, ลบข้อมูล, ล็อกบัญชี) ผ่านทุกโหมด แต่ไม่ได้เก็บเข้า repo

ระวัง ยังไม่มีชุด unit test หรือ e2e อัตโนมัติใน repo และการเดินหน้าจอด้วยมือรอบสุดท้ายหลังแก้โค้ด (T050) ยังไม่ได้ทำ ถ้าถูกถามให้ตอบตรง ขั้นต่อไปคือแปลงตาราง 2.24 เป็น e2e test

เปิดหลักฐาน รายงาน 2.24, สไลด์ 53

Q63 ทดสอบกรณีระบบภายนอกล่มอย่างไร

ตอบ

- หน้า `/dev` มี 11 สวิตช์ตรงกับทางเลือกใน description เช่น 4A RMS ล่ม, 3S RSL ล่ม, 6S Label Service ล่ม, 9A เครื่องพิมพ์, 6A ส่งให้ Supplier ไม่ได้, 5A/10A ฐานข้อมูลบันทึกไม่ได้
- ตัวอย่าง 6A ส่งให้ Supplier ไม่ได้ ต้องได้ “ส่งคำสั่งซื้อไม่สำเร็จ กรุณาตรวจสอบและลองใหม่” และคงสถานะ “รอส่งคำสั่งซื้อ”
- หน้า `/dev` ยังมีปุ่มรีเซ็ตข้อมูลตัวอย่างและปุ่มปลดล็อกบัญชีทั้งหมด (เครื่องมือทดสอบ ไม่ใช่ Use Case)

เปิดหลักฐาน รายงาน 2.24, สไลด์ 53

Q64 ขอดูการทำงานจริงหน่อย ★ ทุกคนต้องตอบได้

วิธีตอบ เปิดเว็บไว้ก่อน (`docker start rsl-pg`, `npm run db:reset`, `npm run dev`) และเปิด psql ค้างไว้อีกหน้าต่าง ทุกขั้น SELECT ก่อนกดและหลังกด (Q65)

ตอบ

1. 1A เข้าสู่ระบบ admin@colorado.jp → SELECT `sessions` ได้ 1 แถวใหม่ (เก็บ token_hash)
2. 4A Dashboard กดนำเข้า Order → “รอตรวจสอบคำสั่งซื้อ” 7 → 8 (toast แจ้งเคสใน seed เช่น ORD-24181 ลูกค้าขอยกเลิก เป็นเรื่องปกติ)
3. 2A ยืนยัน ORD-24194 → ระบบทำ 2S 3S 6S ต่อเอง สถานะ “รอพิมพ์ใบปะสินค้า” `fulfill_source = in_house` ใบปะ ready (INHOUSE-YAMATO-A6)
4. 9A พิมพ์ใบปะ → label printed, `label_print_history` 1 แถว
5. 8A ส่งมอบ → “อยู่ระหว่างจัดส่ง” `delivery_status = in_transit` สต๊อกยังไม่ตัด (7S ตัดหลัง 5S)
6. 5A “คุ้มค่า” ORD-24184 → “รอสั่งซื้อจาก Supplier” (ตาราง reorders ยังไม่เปลี่ยน แถวใหม่เกิดตอนกดสั่งซื้อใน 6A)

เปิดหลักฐาน สไลด์ 54, คู่มือสถาปัตยกรรมหัวข้อเดโม, รายงาน 2.24

> **ระวัง** ทดสอบลำดับนี้แล้วบนข้อมูล `db:reset` ผ่านทุกขั้น ถ้าจะเดโมซ้ำให้กด reset ที่หน้า `/dev` หรือ `npm run db:reset` ก่อน อย่าเปิดหน้า `/shipping` ก่อนเดโม (จะดึงสถานะขนส่งและรัน 6S) บัญชี locked@colorado.jp ล็อก 15 นาทีนับจากเวลารีเซ็ต

Q65 เปิดฐานข้อมูลแล้ว SELECT ให้ดูหน่อยว่าข้อมูลเปลี่ยนจริง ★ ทุกคนต้องตอบได้

ตอบ

1. เปิด DB: `docker exec -it rsl-pg psql -U rsl_app -d rsl_fulfillment_hub` แล้ว `\dt` ดู 17 ตาราง, `\d orders` ดูคอลัมน์, `\x auto` ให้แถวยาวอ่านง่าย

2. ภาพรวม: `SELECT order_status, COUNT(*) FROM orders GROUP BY 1 ORDER BY 2 DESC;`
3. Order ที่ เด โม: `SELECT o.order_id, o.order_status, i.fulfill_source, o.tracking_number, o.delivery_status, l.label_status FROM orders o JOIN order_items i USING (order_id) LEFT JOIN labels l USING (order_id) WHERE o.order_id = 'ORD-24194';`
4. Login: `SELECT admin_id, expires_at, revoked_at FROM sessions;` และ `SELECT user_email, fail_attempts, locked_until FROM admins;`
5. สต๊อก: `SELECT k.sku, s.in_house_qty, s.rsl_qty FROM stock s JOIN skus k USING (sku_id);`
6. ใบสั่งซื้อ: `SELECT reorder_id, sku_id, order_qty, reorder_status, received_qty FROM reorders;`

วิธีตอบ ถ้าอาจารย์ให้เขียนเอง เริ่มจาก `SELECT * FROM <table> LIMIT 5;` แล้วค่อยเติม WHERE / JOIN ถ้าจำชื่อคอลัมน์ไม่ได้ใช้ `\d <table>` ก่อน ห้ามโชว์ `.env.local` หรือรหัสผ่านบนจอ

เปิดหลักฐาน สไลด์ 54, คู่มือสถาปัตยกรรมหัวข้อเดโม

12. ทำไมไม่ทำแบบอื่น ข้อจำกัด และขั้นต่อไป 8 ข้อ

สไลด์ 56

ผู้ตอบหลัก ทุกคน (ผู้ตอบตามหมวดที่เกี่ยวข้อง)

Q66 ทำไมไม่ใช้ Excel หรือเครื่องมือที่มีใน Rakuten RMS อยู่แล้ว

ตอบ

- RMS ดูได้แค่ฝั่ง Rakuten แต่สต๊อกอยู่สองคลังและต้นทุนอยู่ในไฟล์อื่น ปัญหาคือการรวมข้อมูลข้ามระบบ
- Excel ยังต้องคัดลอกข้อมูลเองทุกวัน และสั่งงานต่อเนื่อง เช่น เลือกแหล่งสต๊อกแล้วสร้างใบปะทันทีไม่ได้
- ระบบเราใช้ SKU เป็น key เชื่อมทุกระบบตาม requirement ข้อ 7.2

Q67 ทำไมเขียน SQL เอง ไม่ใช้ ORM

ตอบ

- เกณฑ์ให้แสดง SQL ทุกคำสั่งพร้อมเลข query ถ้าใช้ ORM ต้องเขียน SQL ซ้ำอีกชุดเพื่อทำรายงาน และอาจไม่ตรงกับที่ระบบรันจริง
- เขียนเองเป็นค่าคงที่ใน `src/lib/queries` ใช้ชุดเดียวกันทั้งในเล่ม ในการรันทดสอบ และในเว็บ
- ข้อเสียคือต้องดูแลชนิดข้อมูลเอง

เปิดหลักฐาน 01418321-final-project/src/lib/queries/README.md

Q68 ทำไมเลือก PostgreSQL ไม่ใช้ NoSQL

ตอบ

- ข้อมูลเป็นความสัมพันธ์ชัดเจน สินค้า คำสั่งซื้อ Supplier ใบปะ ต้องมี FK กันข้อมูลกำพร้า
- ต้องการ transaction และ row lock เพราะ action กับ Use Case อัตโนมัติต้องสำเร็จหรือล้มพร้อมกัน และกันตัดสต๊อกซ้ำ
- ใช้ CHECK บังคับช่วงค่าได้ในฐานข้อมูล ตรงกับ range ใน data dictionary

Q69 ทำไมมีบัญชีเดียว ไม่มีพนักงานหลายคน ไม่มี 2FA

ตอบ

- เจ้าของทำงานคนเดียว requirement ข้อ 10 กำหนดให้มีบัญชี Admin ของตัวแทนบริษัท 2FA เลื่อนไปเฟสถัดไปตามข้อ 14
- ข้อมูลตั้งต้นมีบัญชี admin@colorado.jp และ locked@colorado.jp (ใช้ทดสอบกรณีล็อก)

- ถ้าจะมีพนักงานในอนาคต ต้องเพิ่ม role ในตาราง admins และ Use Case จัดการผู้ใช้

Q70 อีเมลที่มีจุด เช่น first.last@ku.ac.th เข้าสู่ระบบได้ไหม

ตอบ

- ไม่ได้ครับ/ค่ะ รูปแบบใน 1A คือ [A-Za-z0-9]+@[A-Za-z0-9]+\.[A-Za-z0-9]+ ไม่รับจุดหรือขีดก่อน @ และโดเมนหลายชั้น
- บัญชีของระบบเรากำหนดเองจึงยังไม่กระทบ แต่เป็นจุดที่ควรแก้เป็นการตรวจรูปแบบอีเมลมาตรฐาน

เปิดหลักฐาน Use Case Description 1A, `src/app/login/actions.ts` EMAIL_PATTERN

Q71 ถ้าจะขยายไป Amazon หรือ Yahoo! Auctions ต้องแก้อะไร

ตอบ

- skus และ orders มีคอลัมน์ sales_channel อยู่แล้ว channel_sku แยกตามช่องทาง จึงเพิ่มสินค้าของช่องทางใหม่เป็นข้อมูลได้
- ที่ต้องเพิ่มคือ adapter ของช่องทางใหม่ตาม interface แบบ RakutenAdapter (ดึง Order, แจ้งเลขติดตาม, ซิงก์สต๊อก) และกฎใบปะของช่องทางนั้น

เปิดหลักฐาน sql/schema.sql, `src/lib/adapters/contracts.ts`

Q72 ข้อจำกัดของงานนี้มีอะไรบ้าง ★ ทุกคนต้องตอบได้

ตอบ

- ระบบภายนอกทั้ง 6 ยังเป็น mock ยังไม่ได้ต่อ API จริงของ Rakuten และ RSL
- ไม่มีการดึง Order ตามรอบเวลา Admin ต้องกดนำเข้าเอง
- ตัวเลขเวลาเป็นค่าประมาณ ยังไม่ได้จับเวลาจริงก่อนและหลังใช้ระบบ
- หน้าจอและ 3S ยังทำงานแบบ 1 รายการสินค้าต่อ Order รับสินค้าได้ครั้งเดียวต่อคำสั่งซื้อ ยังไม่มี Use Case จัดการ Supplier
- ทดสอบด้วยมือตามตาราง 2.24 ยังไม่มี test อัตโนมัติใน repo

เปิดหลักฐาน สไลด์ 56

Q73 ขั้นต่อไปจะทำอะไร

ตอบ

1. ต่อ Rakuten RMS API และ RSL จริงผ่าน adapter แล้วจับเวลางานก่อนและหลังเพื่อวัดตัวชี้วัดใน 1.3
2. เพิ่มการดึง Order ตามรอบเวลา (cron เรียก use-case เดิม)
3. เพิ่ม e2e test จากตาราง 2.24 และ Use Case จัดการ Supplier
4. ขยายไป Yahoo! Auctions และ Amazon

เปิดหลักฐาน สไลด์ 56

13. คำถามถึงตัวบุคคล 4 ข้อ

อาจารย์อาจชี้ถามทีละคน สไลด์ 55

ผู้ตอบหลัก ทุกคน

Q74 ใครเป็นหัวหน้าทีม และคุณทำส่วนไหนในงานนี้

วิธีตอบ ตอบเป็น 3 ส่วน คือชิ้นงานที่ทำ การตัดสินใจหนึ่งเรื่องที่เลือกเอง และผลที่ได้

ตอบ

- ตัวอย่าง “ผมทำ Sequence และ Collaboration Diagram ครับ เลือกให้ทุกเส้นที่ต่อฐานข้อมูลมีเลข query กำกับ ผลคือตามจาก description ไปถึง SQL ที่รันได้ทุกข้อ”

ข้อมูลทีม ตาราง 2.25 กับสไลด์ 55 เติมแล้ว: งานในเล่ม ธนภัทร 16 อชิระ 17 ปิยากร 20 ชิ้น คะแนนจากหัวหน้าทีม 5/5 ทุกคน เกณฑ์ข้อ 36 ให้หัวหน้าเป็นผู้ให้คะแนน หัวหน้าทีมคือธนภัทร

- หัวหน้าทีม: นายธนภัทร พูลโภคา
- ช่วงที่พูด ธนภัทร สไลด์ 1–14 / อชิระ 15–33 / ปิยากร 34–57

Q75 ส่วนไหนยากที่สุด

วิธีตอบ เลือกเรื่องที่เกิดขึ้นจริง และเล่าว่าแก้อย่างไร

ตอบ

- ตัวอย่าง “ยากที่สุดคือทำให้เล่มกับโค้ดตรงกัน ทั้งเลข query, ตาราง และสถานะ เราจึงเก็บ description ไว้ที่ usecases.json ที่เดียว คัด SQL จากโค้ด แล้วสร้าง CRUD, Query Table และผล SQL จากไฟล์นั้น มีสคริปต์เทียบ SQL ในเล่มกับโค้ด”

Q76 ถ้าให้ทำใหม่ จะแก้อะไร หรือได้เรียนรู้อะไร

วิธีตอบ เลือกเรื่องเดียวที่ชัด และเชื่อมกับสิ่งที่เรียนในวิชา

ตอบ

- ตัวอย่าง “จะออกแบบฐานข้อมูลจากโค้ดจริงตั้งแต่ต้น เพราะแบบแรก 15 ตารางต้องปรับเป็น 17 ตารางตอนลงมือทำ”
- หรือ “ได้เห็นว่า CRUD Table ช่วยหา Use Case ที่ขาดได้จริง เช่น ตาราง suppliers ที่ไม่มีใคร Create”

Q77 งานพร้อมนำเสนอครบตามเกณฑ์ไหม

วิธีตอบ เป็นเรื่องที่ต้องเช็กเองก่อนวันนำเสนอ มากกว่าที่อาจารย์จะถาม

ตอบ

- สไลด์ 57 หน้ามีเลขหน้าทุกหน้า รายงานมีทั้ง PDF (297 หน้า) และ Word เว็บเปิดพร้อมเดโม ทุกคนออนไลน์และ**เปิดกล้อง** (ไม่เปิดกล้องหัก 50%)

- เล่มมีปกพร้อมรูป รหัส ชื่อทุกคน (ไม่มีหัก 5 คะแนน) สารบัญ สารบัญรูปภาพ สารบัญตาราง บทคัดย่อไทยและอังกฤษ

14. คำถามรายหน้าเว็บ 7 ข้อ

คู่มือสถาปัตยกรรม บท “ทัวร์หน้าเว็บทีละหน้า” (มีภาพ ปุ่ม Query และ SELECT ทุกหน้า)

ผู้ตอบหลัก นางสาวปิยากร ผลพานิช

Q78 /login และ /dashboard

- **ใส่รหัสผิดหลายครั้ง?** ครบ 5 ครั้งล็อก 15 นาที รหัสเก็บเป็น hash (1A)
- **ดึง Order ตามเวลาไหม?** ไม่ กดปุ่ม “นำเข้า Order” เอง เลข Rakuten ซ้ำจะถูกข้าม (4A)

Q79 /orders/verify

- **กดครั้งเดียวทำไมสถานะกระโดดหลายขั้น?** 2S → 3S (→ 6S) วิ่งต่อเองใน transaction เดียวกัน
- **ลูกค้าขอยกเลิกบน Rakuten?** ระบบถาม Rakuten ซ้ำ ยกเลิกจริง → ยกเลิกแล้ว

Q80 /orders/rsl-match และ /orders/cancel

- **ทำไมต้องจับคู่ RSL?** ต้องได้เลขอ้างอิง RSL ก่อนจึงสร้างใบปะได้ จับคู่แล้ว 6S สร้างใบปะต่อเอง
- **ยกเลิก order ที่ส่งแล้วได้ไหม?** จัดส่งสำเร็จยกเลิกไม่ได้ ระหว่างส่ง → รอดำเนินการพิเศษ ยังไม่ส่ง → คืนสต๊อกถ้าตัดไปแล้ว

Q81 /products, /products/stock, /products/cost

- **กฎ SKU เก็บที่ไหน?** คอลัมน์ `skus.channel_sku` บันทึกแล้ว order ที่ค้างกลับเข้าคิวเอง
- **สต๊อกต่ำแล้วสั่งซื้อให้เองไหม?** ไม่ แค่ขึ้นป้าย และไปโผล่ในแท็บ “รอสั่งซื้อเติมสต๊อก”
- **4S เป็น auto ทำไมต้องกด?** คนกรอกค่า ระบบคิดเลขและเก็บประวัติทุกครั้ง

Q82 /products/reorder

- **กด “คุ้มค่า” แล้วทำไมยังไม่มีใบสั่งซื้อ?** 5A แค่ตัดสินใจ ใบสั่งซื้อเกิดตอนกดสั่งในแท็บ “รอสั่งซื้อเติมสต๊อก” (6A)
- **รับของแล้วเกิดอะไร?** บวกคลังบริษัท order ที่รอใบนั้นกลับไปตรวจสต๊อก (3S) ทันที

Q83 /shipping และ /shipping/label

- **ส่งมอบแล้วทำไมสต๊อกยังไม่ลด?** ตัดตอน 7S หลังแจ้งเลขติดตาม (5S)
- **ติดตามพัสดุตามเวลาไหม?** ไม่มี scheduler ระบบถามขนส่งทุกครั้งที่เปิดหน้า /shipping (เปิดครั้งแรกหลัง reset มี 4 order เปลี่ยนสถานะ)
- **เครื่องพิมพ์เสีย?** หยุดที่กล่องนั้น เก็บประวัติ สถานะไม่เปลี่ยน

Q84 /system/cleanup, /dev, /sitemap

- **ลบ order ที่ยังไม่ปิดได้ไหม?** ไม่ได้ เก็บสำเนาใน `deletion_history` ก่อนลบ
- **/dev คืออะไร?** เครื่องมือทดสอบทางเลือกของ UC (11 สวิตช์) ไม่ใช่ส่วนของระบบจริง สวิตช์มีผลกับทุกคนที่เปิดเว็บ
- **เมนู 11 หน้า แต่ UC 17?** UC อัตโนมัติไม่มีหน้าของตัวเอง 4A อยู่ใน dashboard 5S อยู่ในหน้าจัดส่ง

ตัวเลขที่ต้องจำ

อ่านทุกคน

ตัวเลข	ความหมาย
5--10 ชิ้น/วัน	ยอดขายโดยประมาณ จาก requirement ข้อ 13
12 นาที	งานมือต่อคำสั่งซื้อ 5 งาน (2+3+2+3+2) ค่าประมาณ
1--2 ชม./วัน	60–120 นาที หรือ 30–60 ชั่วโมงต่อเดือน ค่าประมาณ
20--30 นาที → 1 นาที	เวลาคำนวณต้นทุนเดิม กับเป้าหมาย
600 / 200 / 60 เยน	ค่าส่งเอง / ค่าส่งคู่แข่ง / ค่าหยิบของ RSL ต่อชิ้น
2009	ปีที่เริ่มธุรกิจเป็นธุรกิจเสริม
9 → 17	Use Case ก่อนและหลังทบทวน (A 10 ตัว, S 7 ตัว, เพิ่ม 7 ตัว)
17 / 159 / 22	ตาราง / attribute ทั้งหมด / relationship (1:1 สองคู่ ไม่มี M:N)
001--008	ไฟล์ migration ของฐานข้อมูลจริง (008 เพิ่มแค่ sequence เลขใบสั่งซื้อ ตารางยังเป็น 17)
113 SQL	SELECT 50, UPDATE 30, WITH 16, INSERT 15, DELETE 2 รันผ่านทุกข้อ
36 กรณีทดสอบ	ปกติและผิดพลาดทุก Use Case รวม 3S in-house และ 6A รับสินค้า ผ่านทุกกรณี
12 / 2 / 3	หน้าจอ / รายงาน / template ใบปะ
15 / 2	รูป Class Diagram (12 กลุ่มงาน + 3 Entity) / รูป State Diagram
22	ค่าที่เป็นไปได้ของ order_status
6 / 11	adapter ระบบภายนอก (mock) / สวิตช์จำลองข้อผิดพลาดในหน้า /dev
418.61 บาท	ต้นทุนต่อหน่วยของ FSH-MLR-240 (ราคาขาย 1,280)
227.80 บาท	กำไรต่อหน่วยที่คาดของ FSH-RDS-180 ใน 5A (2,690 − 2,462.20)
5 ครั้ง / 15 นาที	เข้าผิดครบแล้วล็อกบัญชี / ระยะเวลาล็อก
8 ชั่วโมง	อายุ session (เก็บแค่ SHA-256 ของ token)
100	จำนวน Order สูงสุดต่อรอบ retry ของ 7S (Q7S.1)
12 เดือน	ลบตามช่วงวันที่ได้เฉพาะข้อมูลที่เก่ากว่านี้ (10A)

เกณฑ์ให้คะแนนกับคำถาม

อ่านทุกคน

เกณฑ์แต่ละข้อน่าจะโดนถามที่ Q ไหน (จาก “รายละเอียดของคะแนนโครงงาน ฉบับ update”)

ข้อ	เกณฑ์	หัวข้อในเล่ม / สไลด์	คำถาม
--	เปิดการนำเสนอด้วยภาษาอังกฤษ	สไลด์ 1, 3	Q1
--	ตารางแบ่งความรับผิดชอบ	2.25, สไลด์ 55	Q74
0--4	ปก สารบัญ รูป ตาราง บทคัดย่อ	ส่วนหน้าของเล่ม	Q77
5	บทนำ แนะนำหน่วยงาน	1.1, สไลด์ 3	Q1, Q2
6	ปัญหาพร้อมตัวเลข	1.2, สไลด์ 4	Q3, Q6
7--8	BP เดิมและคำอธิบาย	2.1–2.2, สไลด์ 7	Q9
9	ปัญหา ของ BP วัดได้ ไม่ใช่ “ไม่ใช้คอมพิวเตอร์”	2.3, สไลด์ 8	Q4, Q10, Q5
10--11	BP ใหม่และคำอธิบาย	2.4–2.5, สไลด์ 9–10	Q11, Q12
12--13	BP ที่สัมพันธ์กับ UC และตารางจับคู่	2.6–2.7, สไลด์ 11	Q13, Q14
14	UC ก่อนปรับปรุง actor ตาม swim lane	2.8, สไลด์ 12	Q19, Q16
15--16	CRUD และคำอธิบายว่าขาด UC อะไร	2.9–2.10, สไลด์ 12	Q20, Q17
17--18	UC และ CRUD ที่ทบทวนแล้ว	2.11–2.12, สไลด์ 13–14	Q17, Q18, Q21, Q22
19	UC description สอดคล้องกับ UI และ ER	2.13, สไลด์ 15–33	Q23, Q26, Q27, Q24, Q25
20	ER: PK/FK, ไม่มี M:N, participation	2.14, สไลด์ 34	Q36, Q37, Q38, Q40
21	ER พร้อม query	2.15, สไลด์ 35	Q39
22	Query Table	2.16, สไลด์ 36	Q15, Q59
23	Table Structure	2.17, สไลด์ 36	Q41
24	Sequence: MVC หน้าจอติด actor ไม่ส่งกลับ actor, DB connect ใช้จริง	2.13, สไลด์ 17, 21, 26, 29, 32	Q30, Q31, Q32, Q34, Q35
25	Collaboration ตรงกับ Sequence มีเลข	2.13, สไลด์ 22, 27, 30	Q33
26	Class: ทุก object เป็นคลาส ความสัมพันธ์	2.18, สไลด์ 37	Q42, Q43
27	State: class.state, [condition]event/UC, SQL	2.19, สไลด์ 38–43	Q44, Q45
29	SW development platform	2.20, สไลด์ 44–49	Q53, Q54, Q46, Q47

ข้อ	เกณฑ์	หัวข้อในเล่ม / สไลด์	คำถาม
30	Site map root คือ login	2.21, สไลด์ 50	Q55
31	UI ทุกหน้าจอ ตั้งชื่อตาม UC	2.21, สไลด์ 50	Q55
32	Output design (report)	2.22, สไลด์ 51	Q56, Q57
33	SQL ทุกคำสั่งรันได้ ไม่ใช้ AI generate	2.23, สไลด์ 52	Q58, Q59
34	ทดสอบตาม UC พร้อมรูปหน้าจอและรายงาน	2.24, สไลด์ 53	Q62, Q63, Q61
35	ความพร้อม สไลด์ รายงาน โปรแกรม กล้อง	สไลด์ 54	Q77, Q64, Q65
36	คะแนนการทำงานโดยหัวหน้า	2.25, สไลด์ 55	Q74

เกณฑ์ข้อ 28 ไม่มีในไฟล์ (เลขข้ามจาก 27 ไป 29) คำถามเชิงเทคนิคเพิ่มเติม Q49, Q50, Q51, Q52, Q48 อยู่ในหมวด 8